Empirical Software Engineering (2026) 31:21
https://doi.org/10.1007/s10664-025-10744-9

An efficient model maintenance approach for MLOps

Forough Majidi[1] · **Foutse Khomh**[1] · **Heng Li**[1] · **Amin Nikanjam**[1]

Received: 3 December 2024 / Accepted: 24 September 2025 / Published online: 17 November 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
In recent years, many industries have utilized machine learning (ML) models in their systems. Ideally, ML models should be trained on and applied to data from the same distributions. However, the data evolves over time in many application areas, leading to concept drift, which in turn causes the performance of the ML models to degrade over time. Therefore, maintaining up-to-date ML models plays a critical role in the MLOps pipeline. Existing ML model maintenance approaches are often computationally resource-intensive, costly, time-consuming, and model-dependent. Thus, we propose an improved MLOps pipeline, a new model maintenance approach and a Similarity-Based Model Reuse (SimReuse) tool to address the challenges of ML model maintenance. We identify seasonal and recurrent data distribution patterns in time series datasets throughout a preliminary study. Recurrent data distribution patterns enable us to reuse previously trained models for similar distributions in the future, thus avoiding frequent unnecessary retrainings. Then, we integrated the model reuse approach into the MLOps pipeline and proposed our improved MLOps pipeline. Furthermore, we develop SimReuse, a tool to implement the new components of our MLOps pipeline to store models and reuse them for inference of data segments with similar data distributions in the future. Our evaluation results on five time series datasets demonstrate that our model reuse approach can maintain the models' performance while significantly reducing maintenance time, costs, and the number of retrainings. Our model reuse approach achieves ML model performance comparable to the best baselines, while reducing the computation time and costs by 87.5%. Therefore, industries and practitioners can benefit from our approach and use our tool to maintain their ML models' performance in the deployment phase to reduce their maintenance time and costs.

Keywords MLOps · Model maintenance · Model monitoring · AIOps · Machine learning model lifecycle · Model reuse · Concept drift adaptation · Machine learning engineering

Communicated by: Foutse Khomh and Bowen Xu

This article belongs to the Topical Collection: *Special Issue on SEA4DQ*.

Extended author information available on the last page of the article

1 Introduction

Many companies and industries have started using machine learning (ML) in their businesses or replacing their traditional non-ML-based software systems with ML-based ones (Brynjolfsson and Mcafee 2017). These industries train an ML model using a training dataset and then use it to make predictions. However, data is not stationary in many fields and changes over time due to concept drift. When this happens, the trained model may not work well with the new data, leading to poor performance. As a result, maintaining up-to-date ML models in the production environment is essential, as their performance can degrade over time due to concept drift (Bayramet al. 2022).

MLOps pipelines are employed to develop, deploy, and maintain ML-based software systems over time (Spotify Engineering 2023; Hopsworks 2023; Sculley et al. 2015). Such pipelines typically contain components like model "model performance monitoring" and "trigger" to train a new model when the performance drops below a specific threshold (Google Cloud 2024). However, using ML in software systems has its own challenges (CircleCI 2024). For instance, the increasing size of datasets and the growing complexity of ML models make training a new model and maintaining ML-based software systems progressively more difficult, resource-intensive and time-consuming (CircleCI 2024). For instance, training of a LSTM to forecast how many boxes each Coca-Cola client will buy next month takes 100 hours per training (100 epoch) and costs US$15,000 for 10 training AWS ml.p3.8xlarge instance (AnalyticsVidhya 2020)

In real-world, a lot of industries and applications (e.g. energy consumption) use time series datasets and this type of data may show seasonality (i.e., periodic changes such as daily, weekly, or yearly cycles) and reocurring patterns (i.e., repeated data distributions over time). These characteristics are common in time series datasets and can be leveraged to better understand and manage changes in data distribution (Hyndman and Athanasopoulos 2018; Shumway and Stoffer 2017). However, despite their importance, to the best of our knowledge, these patterns have not been used in the context of model maintenance in MLOps.

The literature proposes several approaches, such as periodic model retraining, to support the maintenance of ML-based software systems and ML models in production environments (Lyuet al. 2023). Although these approaches aid in maintaining high prediction accuracy and user satisfaction, each method has its own set of limitations, highlighting the need for a more effective and comprehensive solution. For example, periodical model training is one of the best maintenance approaches that mostly achieves high performance compared to other approaches (Lyuet al. 2024, 2021a). While increasing the frequency of model retraining can improve the performance (Lyuet al. 2021a), retraining is costly, time-consuming, and requires expensive hardware resources. To the best of our knowledge, none of the prior work has examined recurrent and seasonal data distributions. Leveraging such patterns to reuse previously trained models could reduce the cost and time of ML model maintenance. Furthermore, other model maintenance approaches like online learning (Lyuet al. 2024) are model-dependent and cannot be applied to all types of models. Therefore, this research aims to 1) explore the seasonality and recurrent drifts in the time series data by conducting a preliminary study and 2) provide a novel model-agnostic maintenance approach that maintains ML models in production while keeping resource usage, maintenance time, and cost low, and 3) evaluate the performance and cost of the proposed method. In this study, we present

a tool named SimReuse to save models and use them again later to make predictions on data with a similar data distribution to the training data.

Our study is organized by conducting a preliminary study, proposing our maintenance approach and then answering two Research Questions (RQs) to evaluate our approach:

Preliminary Study Time series datasets are common in many application areas like the stock market price (Timescale Contributors 2024). The preliminary study aims to understand whether recurrent and similar seasonal distribution patterns exist in the time series datasets. The goal is to understand the characteristics of data relevant to model maintenance. First, we analyze the distribution of time series datasets over time to check if we find any recurrent and seasonal distribution patterns. Then, we split the datasets into specific segments and use similarity measurement techniques to find similar distributions over time. As a result, we observed that recurrent patterns exist. Afterward, we attempt to improve the MLOps pipeline (Google Cloud 2024) by adding new components to benefit from these recurrent patterns for reusing our previously trained models. Our improved MLOps pipeline reuses the previously trained models for similar data distributions in the future. This new model maintenance approach reduces computation resources, time, and maintenance costs. Developers can set the parameters of SimReuse and use this tool for their projects. To assess the effectiveness of SimReuse, we investigate the following research questions:

RQ1: How does SimReuse perform regarding ML performance metrics? Keeping the performance high is one of the critical factors for practitioners when choosing an ML model maintenance approach. We evaluate the performance impact of our approach by comparing it to our baseline approaches to ensure it works well in the production environment. The results show that our model reuse approach performs comparably to the periodically updated models and informed retrained models while outperforming the stationary and random output models.

RQ2: How does SimReuse perform regarding maintenance costs? Training a model is costly for companies (Lyuet al. 2021a). For instance, if a company uses AWS SageMaker with the "ml.p4de.24xlarge" instance, training a model that takes 10 hours costs 370$, which is quite expensive if the training takes longer. Our approach aims to reduce the cost of model maintenance by reusing the existing models while keeping the performance high. Therefore, in RQ2, we evaluate the cost of our model reuse approach and compare it with the cost of baseline approaches. Our results show that SimReuse reduces maintenance time and cost by 87.5% and 11.5% compared to the periodical retraining and informed model update approaches, respectively.

To the best of our knowledge, SimReuse is the first tool that proposed the following contributions to the existing model maintenance techniques:

– Our improved MLOps pipeline and model reuse approach are the first model maintenance solutions that benefit from recurring data distributions and seasonalities in time series datasets to reuse the previously trained models in the ML-based software systems. Furthermore, our approach is model- and application-agnostic, making it applicable to any ML model and time-based dataset.

- We reduce the frequency of model retraining and associated model maintenance costs by utilizing existing models trained on distributions similar to the upcoming distribution, while maintaining performance comparable to the baseline models.
- We demonstrate the effectiveness of our model maintenance approach from the aspects of ML performance metrics and model maintenance costs using five real-world time series datasets. Our approach not only matches the ML performance of the best baseline models but also greatly lowers the costs associated with maintaining the model.
- We forecast the future distribution and proactively choose the ML model before a new data segment arrives.

The remainder of this paper is organized as follows: Section 2 provides background information. Section 3 reviews the related works and positions our work relative to them. Section 4 conducts a preliminary study on the characteristics of the datasets. Section 5 proposes our improved MLOps pipeline and our model reuse approach. Section 6 explains our evaluation process and presents the results for answering our research questions. Section 7 explains the implications of our work for different groups of people. Section 8 discusses the threats to the validity of our study. Finally, section 9 concludes this work and proposes ideas for future works.

2 Background

This section explains the necessary background knowledge.

2.1 MLOps Pipeline

This research introduces an enhanced MLOps pipeline by incorporating new components. Before introducing our new components, we first present an overview of the MLOps pipeline and briefly describe its key elements.

An MLOps pipeline helps manage machine learning projects by automating critical tasks like getting data ready, training models, and deploying them. It helps teams to work more efficiently and faster by using tools/approaches that automatically check the new data, detect drift, retrain models with new data, test them, and make them ready for use. The main goal is to keep the ML system running smoothly and ensure that models can be updated easily when needed. Mixing machine learning with DevOps methods enables MLOps to make machine learning systems more reliable, efficient, and able to grow (Google Cloud 2024; RunAI Contributors 2024). Also, Automated Machine Learning (AutoML) tools can be used to automate one or multiple components of the MLOps pipeline (e.g. model training) (Majidi et al. 2022). Figure 1 shows the MLOps pipeline (Google Cloud 2024).

The explanation of the MLOps components based on the Google definitions (Google Cloud 2024) is as follows:

Feature Store feature store is an optional component, and it serves as a centralized repository that standardizes the definition, storage, and retrieval of features for both training and serving.

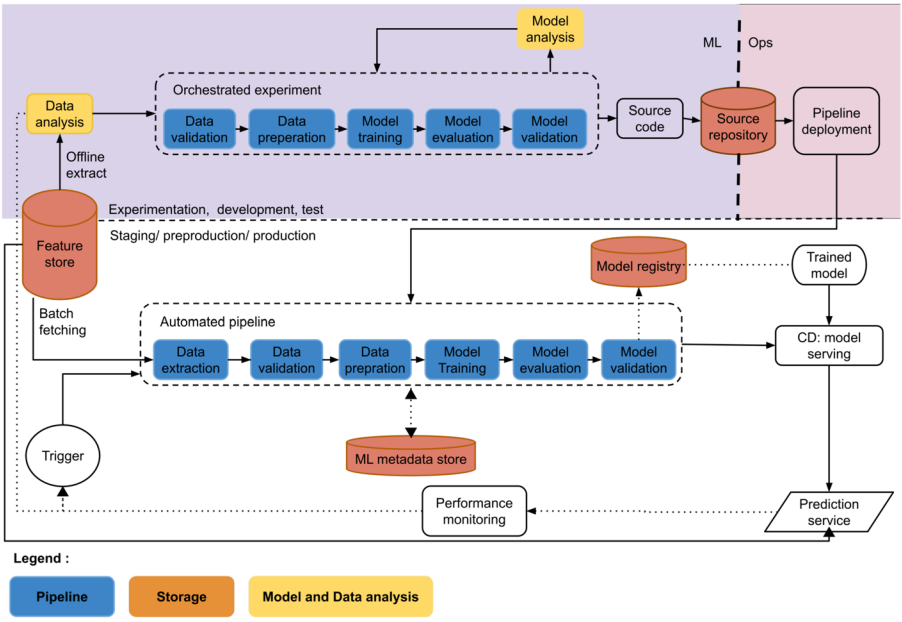


Fig. 1 MLOps pipeline (Google Cloud 2024)

Data Analysis Data analysis involves performing exploratory data analysis (EDA) to understand the structure and key details of the data to build the ML model. This step helps identify the necessary data preparation and feature engineering and ensures the data meets the model's expectations.

Data Validation This step is needed before training the model to decide if you should retrain the model or stop the pipeline. The pipeline automatically makes this decision if specific issues are found. The issues are Data schema skews and Data values skews.

Data Preparation This step is needed before training the model to preprocess data. Preprocessing data includes steps like data transformation, data normalization, data cleaning, and so on. The output of this step is preprocessed data.

Model Training In this step, the prepared data is used to apply different algorithms and train various ML models. The algorithm is also fine-tuned using hyperparameter tuning to find the best model. The result of this step is a trained model.

Model Evaluation The model is evaluated using the test set data. The output of this step is the results of quality assessment metrics such as the F1 score, accuracy, precision, mean squared error, and so on.

Model Validation The model is approved to be suitable for deployment, and its performance is better than baselines.

Model Analysis This component provides a deeper inspection of the trained model, extending beyond performance metrics to focus on aspects such as robustness, security, explainability, interpretability, and bias detection.

Source Code This component includes the source code.

Source Repository The source code is stored in the source repository.

Pipeline Deployment The whole training pipeline is deployed. Then, the trained model serves as the prediction service. ML practitioners can deploy their machine learning projects on Docker (Openja et al. 2022).

Model Registry The trained model is stored in the model registry.

Trained Model This component contains the trained model.

Model Serving The validated model is set up in a production environment to provide predictions.

Data Extraction The needed data from different sources is combined.

Prediction Service The model is used to make predictions on the deployment environment's data.

ML Metadata Store This component records the important information, including the execution time, parameter arguments, and so on.

Performance Monitoring The performance of the ML model in the production environment is checked, and an operation is triggered when the performance drops below a certain threshold.

Trigger When the model's performance in the production environment drops below the threshold, a few operations can be triggered, including 1) going back to the development phase and executing a new experiment cycle and 2) executing the pipeline.

Retraining ML models can be triggered by on-demand execution, on a schedule of routine updates, or occasional updates when new data is available. It may also occur due to declining performance or notable changes in data distributions (concept drift).

2.2 Time Series Data and Seasonality

Time series data consists of a collection of observations of a subject over time intervals (Clarifyio Contributors 2024). Many domains have time series data. For instance, Weather forecasting, demand forecasting, traffic flow prediction, and so on are all time series datasets. Many time series datasets (e.g. economics and business) contain seasonal patterns (Zhang and Qi 2005). Seasonality refers to recurring changes/distribution patterns that are caused

by factors like days of the week, weather, holidays, months of the year, and so on Zhang and Qi (2005). This study proposes that if distribution patterns in the dataset repeat, we can reuse models trained on similar data.

2.3 Machine Learning Algorithms

This section explains the machine learning training algorithms that we utilize in this research. These algorithms include classical algorithms such as Random Forest (RF) and XGBoost (XGB), as well as deep learning algorithms such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU).

The description of the four algorithms used in this research is provided below.

- **Random Forest (RF)**: Random decision forest (RF) is an ensemble learning method that builds multiple decision trees during training and can be used for regression and classification tasks. The main goal of RF is to combine the output (predictions) of these trees to improve the robustness and performance of the model. In RF, each tree is trained on a randomly split subset of data, and the results of all trees are combined at the end using averaging for regression and majority voting for classification to generate the final result. The RF improves the generalizations and reduces the overfitting (IBM 2024; Wikipedia Contributors 2024b; Günay 2021).
- **XGBoost (XGB)**: eXtreme Gradient Boosting (XGB) is a machine learning model that uses ensemble learning and is suitable for regression and classification tasks (Jafari et al. 2025). XGB implements gradient-boosted decision trees designed for speed and performance. The XGB combines the prediction of multiple models (e.g. decision trees) iteratively to build a predictive model. It incrementally adds weak models (trees) to the ensemble, with each new model emphasizing more on the data points that were previously misclassified. Also, XGB prevents overfitting by regularization (Vidhya 2018; NVIDIA 2024).
- **Long Short-Term Memory (LSTM)**: LSTM is a special neural network structure that is designed to remember the relevant information for long periods when dealing with sequences like time series data, text, or speech. Unlike conventional recurrent neural networks that often confuse or "forget" details in long sequences, LSTMs use a set of "gates" to determine which pieces of information should be kept, updated, or removed from the network over time. This approach ensures that critical events at the start of the sequence can still be used when making predictions much later. The main strength of LSTMs is that they separate their "cell state" (which handles long-term memory) from a short- term hidden state, helping them avoid issues like exploding gradients or vanishing. The design choice of LSTMs helps them succeed in applications like predicting future trends in a time series, predicting words in a sequence, and translating between languages (Hochreiter and Schmidhuber 1997; Gers et al. 2000).
- **Gated Recurrent Unit (GRU)**: GRUs are specialized types of recurrent neural networks designed to handle sequential information like time series, text, and audio by keeping track of important information over several time steps. They achieve this through using "gates" which act like regulators that decide how much new input to keep and how much old data to discard. At each point in a sequence, the GRU decides what new information should be added to the memory and what old information should be deleted. This

design helps GRUs remember key details for a long time and discard irrelevant ones. It also helps GRUs avoid issues seen in traditional RNNs, such as "forgetting" significant parts of the sequence or having difficulty in training because of exploding or vanishing gradients (Goodfellow et al. 2016; Chunget al. 2014).

We utilize RF because it is effective for complex relationships and high-dimensional datasets (Undy 2024). Since our datasets are multivariate, RF can capture the relationships between different features. RF also reduces overfitting by averaging the results of multiple decision trees. Besides, RF is scalable and robust enough to handle outliers (Undy 2024).

We utilize XGB because it is good at capturing non-linear relationships. Identifying non-linear relationships is important for time series datasets, where patterns may not always be linear. It applies regularization to avoid overfitting. XGB is also scalable and able to handle outliers (Undy 2024). Both RF and XGB can handle multiple input variables (Undy 2024).

We use LSTM and GRU because these models are popular models used for time series forecasting (Goodfellow et al. 2016; Hochreiter and Schmidhuber 1997; Chunget al. 2014).

2.4 Concept Drift

In machine learning, when the distribution of the target concept changes over time, there is a "concept drift" (Nishida and Yamauchi 2007; Widmer and Kubat 1996; Gama et al. 2014). Concept drift and performance degradation are closely related phenomena in machine learning systems. The presence of one implies the existence of the other (Bayramet al. 2022). Concept drift can render models trained on earlier data obsolete, leading to a decline in performance (Lyuet al. 2021a). Reoccurring concept drift is a type of concept drift that refers to the reappearance of concepts over time (Zliobaite 2010). This research targets reoccurring concept drifts.

2.5 Evaluation Metrics

This section explains the metrics we used to evaluate our approach.

Performance Measurement Metric Our datasets are time series data with numeric target values. We use 1) Mean Squared Error (MSE) (Wikipedia Contributors 2024a) and 2) Mean Absolute Error (MAE) as measurement metrics to evaluate the performance of the machine learning models. MSE computes the average squared distance between the actual value and the predicted values. The MSE is given by (1). MAE computes how far the predictions are from the actual values on average (Willmott and Matsuura 2005). The MAE is given by (2).

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{1}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2}$$

Statistical Comparison Metric We aim to compare our approach's performance with the baseline approaches' performance. Therefore, we use a non-parametric pairwise statistical test to compare the results. Specifically, we use the Mann-Whitney U statistical test to compare the results of our approach with those of the baseline approach. We choose Mann-Whitney U because it is a non-parametric test that does not assume that the data is normally distributed (which is not true in the case of our data). Also, this test is suitable when having a small sample size, which is true in our case. Finally, this test is appropriate when the samples are independent (each row of results is independent of the other rows of results).

The threshold we picked to separate significant and insignificant differences is 0.05 for the p-value (Lyuet al. 2021a). The p-values greater or equal to 0.05 show an insignificant difference between the results of the two compared approaches. The p-values less than 0.05 show a significant difference between the results of the two approaches.

Estimated Maintenance Time and Cost we use operation time as metrics for model maintenance costs. These times are calculated using (3), where "operation" refers to the processes of training, prediction, forecasting, and similarity measurement.

$$\text{Operation time} = \text{operation end time} - \text{operation start time} \tag{3}$$

In addition, we calculate the financial cost of our approach and compare it to the financial cost of baseline approaches. We run the codes using a Macbook Pro version 2021 with an Apple M1 Pro chip, eight cores (six performance and two efficiency), and 16GB memory, and MacOS Ventura 13.1. To measure the financial cost of running the code, we measure the cost of computations using the equivalent AWS SageMaker instance. The equivalent AWS Sagemaker instance to our MacBook Pro specifications is ml.m5.large (AmazonWebServices 2024a). The hourly rate of this instance is $0.115. To calculate how much our computations cost, we use the (4).

$$\text{Financial Cost} = \text{Hourly Rate} \times \frac{Minutes}{60} \tag{4}$$

3 Related Works

In this section, we first briefly review related prior work and highlight their limitations.

3.1 Impacts of Concept on Model Performance

Concept drift is a common challenge that companies face when deploying their models. For instance, Lyuet al. (2021a) shows that concept drift exists in the AIOps datasets like the Google cluster trace dataset (Wilke 2020) and the Backblaze disk stats dataset (Backblaze 2024). Concept drift can lead to model performance degradation because the models have not seen the new data distributions and are not trained for the new patterns. Therefore, model maintenance plays a critical role in handling concept drifts in production.

There are different types of drift that can occur in the data. Zliobaite (2010) discusses four different types of drift that may occur in the dataset, as shown in Fig. 2. In sudden drift, the distribution of data shifts instantly at a certain point in time. In gradual drift, the data distribution changes slowly from one concept to another, with overlapping phases. In incremental drift, the data distribution changes step-by-step over time in a continuous manner. In reoccurring contexts (reoccurring concept drift), previously observed data distributions reappear periodically or sporadically over time. In this study, we examine the reocurrence of data distribution and seasonalities in time series datasets over time. Thus, we target the reoccurring concept drifts (existence of reoccurring contexts) of the time series datasets (UbiOps Contributors 2024). By reocurring drift, we refer to situations where the same contexts reappear over time, causing repeated changes in the data.

In subsections 3.2, we explain the model maintenance and drift mitigation approaches proposed by the literature and their shortcomings.

3.2 Model Maintenance and Concept Drift Mitigation

In this section, we explain the proposed model maintenance approaches and the efforts made to compare different approaches. We also explain the studies that tried to mitigate drift.

Lyu et al. (2021a) show a method to maintain and update models that are deployed in production to reduce the risk of concept drift. They presented that periodically updating models can mitigate the model performance degradation and concept drift issues, outperforming the performance of the stationary model (the initial model never gets updated in the stationary model approach). However, periodically retraining the model is more costly than stationary models (Lyuet al. 2021a).

Poenaru-Olaru et al. (2023) propose an automated maintenance pipeline for AIOps models that detects concept drift and selects the most suitable model retraining method based on the type of drift detected. Poenaru-Olaru et al. (2022) show that class imbalance, which is unavoidable in operational data, negatively impacts the accuracy of the concept drift detectors. Although they limited the model training to the number of drifts in the dataset, they did not reuse the existing models.

Previous works also mention two other types of model maintenance and retraining techniques: “full history” and “sliding window” (Lyuet al. 2021b; Poenaru-Olaru et al. 2024). The full history approach continuously combines the training dataset with the recent data and retrains the model. The sliding window method retrains the model with only the most recent data as the training data while discarding old samples.

Although both full history and sliding window approaches achieve good results, based on the specific scenario, one approach may perform better than the other (Lyuet al. 2021a; Poe-

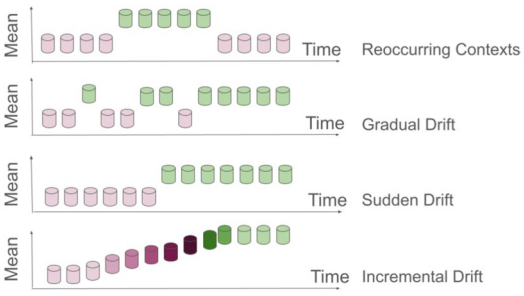


Fig. 2 Four different types of drift (Zliobaite 2010)

naru-Olaru et al. 2023). Sometimes, the drift occurs seasonally. For example, an increase in the usage of servers may cause more job failures. Server usage might rise during certain times, leading to an increase in job failures. In this example, the full history method is better because the full history is trained on data that captures multiple periods and distributions. However, the sliding window method may not have seen all the seasonal distributions (Poenaru-Olaru et al. 2023).

Lyu et al. (2024) conducted an empirical study of five different model update approaches for supervised learning, evaluating them in terms of stability, update costs, and performance. Their results demonstrate that concept drift-guided retraining, periodic retraining, online learning, and time-based model ensembles offer superior performance and greater stability compared to stationary models. However, their findings also indicate that, despite better performance, online learning and time-based ensembles increase model testing time. Consequently, their use in AIOps applications is limited, as these applications require immediate inferences.

To mitigate the impact of concept drift on model performance, some research papers propose methods for detecting (Gama et al. 2004; Harel et al. 2014; Nishida and Yamauchi 2007; Wang et al. 2013) and handling concept drift (Brzezinski and Stefanowski 2013; Cano and Krawczy 2020; Dongre and Malik 2014; Gama et al. 2014; Hoenset al. 2012; Minkuet al. 2009; Minku and Yao 2011; Street and Kim 2001; Wang et al. 2003). Poenaru-Olaru et al. (2024) propose a concept drift mitigation technique for anomaly detection systems called "Informed model update". They suggest that instead of blindly updating anomaly detection models, it is better to update them whenever a change is detected in the data. This method helps to avoid unnecessary retraining.

Mallick et al. (2022) propose a flexible, adaptable, and scalable data drift mitigation solution (Matchmaker) for deploying large-scale systems. Some of the differences between our approach and the Matchmaker are 1) Matchmaker focuses on large-scale systems, while our approach can handle any time series datasets with seasonality and reoccurring patterns, 2) Matchmaker identifies similar batches for each test point, while our approach uses the overall distribution similarity for each data segment, reducing the inference time, 3) MatchMaker reactively chooses the correct model when a new data instance arrives, which causes additional latency introduced by comparing the new instance with existing data, whereas our approach proactively forecasts the distribution in the future and proactively chooses the model. Therefore, MatchMaker has more overhead compared to our approach. Table 1 shows the other differences between Matchmaker and SimReuse. Our model reuse approach clusters the data based on the seasonality and similar data distributions, while Matchmaker does not consider the distribution of data while clustering it. Our approach focuses on the

Table 1 Differences between Matchmaker and SimReuse

Features	SimReuse	Matchmaker
Clusters the data based on seasonal/recurrent patterns or similar distributions	✓	✗
Proactive approach	✓	✗
Data clusters represent similar sets of data distributions	✓	✗
Reduces the frequency of periodically updating models	✓	✗
Focuses on time series datasets with reoccurring and seasonal drifts	✓	✗
Relies on offline computations	✓	✗

reocurring and seasonal drifts, while it is not the case in Matchmaker approach. Also, our approach relies on offline computations.

In summary, the aforementioned prior works have the following shortcomings:

- The current model maintenance techniques, such as periodically updating the model, are not efficient enough in terms of time and cost of training and testing. Some of the existing approaches (e.g. online learning) are also model-dependent.
- To the best of our knowledge, none of the previous studies have considered the reocurring and seasonal patterns in data distributions to leverage existing models and avoid unnecessary retraining.
- Existing approaches detect drift when the new data arrives, but we are predicting the future distribution and proactively choosing the ML model. Some prior works use the existing data in their approach, but it is not realistic in the production environment because we do not have future data.

In this study, we aim to propose a new model maintenance approach to address model maintenance in case of reoccurring concept drift. We also propose an improved MLOps pipeline that developers can use when creating their own MLOps pipeline.

4 Preliminary Study

In this section, we conduct a preliminary study to analyze the distribution patterns in the datasets, aiming to discover whether recurrent concept drifts and repeating seasonal patterns exist in the time series datasets.

Figure 3 presents the overview of the preliminary study design. First, we choose the datasets and preprocess them. Next, we find the right window size to divide each dataset into smaller segments. Then, we use forecasting methods to predict what the data distribution will look like in the future in the production environment, and we use similarity measurement methods to find which past data window it is most similar to. Finally, we study the similarities between the forecasted data segments and the earlier ones. The process of dataset selection and design of the preliminary study is explained in more detail below.

4.1 Datasets

To select our datasets, we applied the following criteria: the datasets should 1) be real, not synthetic data; 2) be used in other research papers, 3) span at least one year, and 4) be multivariate time series. Five public multivariate time series datasets are selected for this

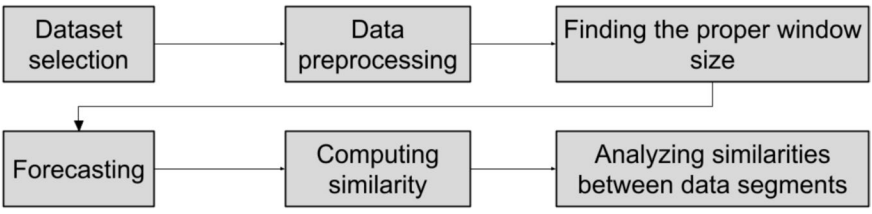


Fig. 3 Overview of the preliminary study

study based on the criteria. The datasets are described below. Also, more information about the start date, end date, number of total samples, number of features, number of samples per day, and time interval between the samples is provided in Table 2. We chose these five datasets because they are real datasets, and they are widely used in other research papers like (Zliobaite 2013; Mallick et al. 2022; Das et al. 2023; Jin et al. 2023; Patil et al. 2024). It is worth mentioning that Electricity Zurich, Oil temperature, and Weather datasets are selected among the eight real-time series datasets that the Dart Python library provides (Herzen et al. 2022).

- **NSW Electricity** (Neto andCanuto 2021): The electricity dataset includes data gathered from the New South Wales Electricity Market in Australia, where electricity prices are not fixed. Instead, prices fluctuate based on demand and supply over the last five minutes. The task is to determine the electricity price of the New South Wales (NSW) city in Australia based on other features (Neto andCanuto 2021). The electricity dataset is influenced by concept drift because of shifting consumption patterns, unforeseen events, and seasonal variations. It has been used in over 40 concept drift experiments (Zliobaite 2013). Prior studies have used the electricity dataset to evaluate their proposed approaches that target datasets with concept drift (Mallick et al. 2022; Neto andCanuto 2021; Abbasi et al. 2021). The features of this dataset are also normalized between zero and one.
- **Zurich Electricity** (Herzen et al. 2022; Zürich 2022, 2025): The Zurich Electricity dataset includes electricity usage by businesses and services (medium voltage), as well as by households and small-to-medium enterprises (SMEs) (low voltage), in the city of Zürich (2022). The electricity consumption data is integrated with hourly weather measurements from three stations in Zürich (2025). The original weather data, recorded hourly, is resampled to a 15-minute frequency, with any missing values interpolated before merging with the electricity consumption data (Unit8co 2024). The task here is to forecast the electricity consumption of Households and SMEs. More description about the features of this dataset is provided in Unit8co (2024).
- **Oil temperature** (Herzen et al. 2022; Zhou 2024; Zhou et al. 2021): This dataset includes the data of one electricity transformer at one station (Unit8co 2024). The task here is to forecast the oil temperature. More description about the features of the dataset in provided in Unit8co (2024).
- **Weather** (Herzen et al. 2022; Max Planck Institute for Biogeochemistry 2024; Zenget al. 2023): This dataset, recorded every 10 minutes, includes 21 indicators of the weather in Germany like humidity and air temperature (Unit8co 2024). The task here is to forecast the weather temperature.
- **NIFTY-50 Stock Market** (Patil et al. 2024; Jainet al. 2018): This dataset includes the stock price history and the trading volumes of the fifty stocks in NIFTY-50 index from National Stock Exchange (NSE) India. The task here is to forecast the close price. Close price is the last agreed-upon value between sellers and buyers before the market closes for the day.

4.2 Design of the Preliminary Study

This section explains how the preliminary study is conducted.

Table 2 Additional details of the datasets

Dataset	Start date	End date	No. of total samples	No. of features	No. of samples per day	Time interval between the samples	Target value
Electricity NSW	1996-06-3	1998-12-05	4,4016	4	48	30 min	NSW electricity price
Zurich Electricity	2020-01-01	2020-12-31	35,040	9	96	15 min	Zurich electricity consumption
Oil temperature	2016-07-01	2018-06-25	17,400	6	24	60 min	Oil temperature
Weather	2020-01-01	2020-12-31	52,704	20	144	10 min	Weather temperature
NIFTY-50	2008-1-3	2020-12-30	3,198	10	1	24 hour	Close price

Data Preprocessing To preprocess the data, all features of all datasets are normalized to values in the range of [0, 1] using the Min-Max scaler algorithm implemented by the scikit-learn library (Scikit-learn Developers 2024), ensuring comparable feature contributions. Additionally, all dates are converted to the *datetime* format using the Pandas *datetime* function (The Pandas Development Team 2024).

Finding the Proper Window Size We aim to train models on data windows with different data distributions and reuse the models in the future. One of the steps to achieve this goal is to find the proper window length to segment the data. To do so, we use the Autocorrelation Function (ACF) to find the reoccurring patterns in the data distribution.

The Autocorrelation Function (ACF) is a statistical tool that quantitatively measures the similarity between the time series and the lagged versions of itself over varying time intervals (lags). By comparing the original series with lagged versions, the correlation at each lag indicates how much past values resemble current values. Strong correlations at particular lags often indicate the presence of repeating behavior or cycles within the data (Boxet al. 2015). For example, if a time series follows a repeating pattern with a particular period p, the autocorrelation will be higher at a lag of p. In other words, the values of the series at time t are more similar to the values of the series at time t - p when there exists a repeating pattern every p steps.

The main application of the ACF in time series analysis is detecting seasonalities. If the ACF detects high similarity at specific intervals, it represents the presence of recurring and consistent behaviors at those time intervals. For instance, when analyzing daily data, strong correspondence at intervals of 14 days often represents a biweekly pattern. This information is critical for building predictive models like resource planning, sales forecasting, and so on. Repeated rhythms significantly impact the outcome of predictive models (Hyndman and Athanasopoulos 2018). In this research, we utilize ACF to assess if there are reoccurring patterns in the evaluated time series datasets.

Our Application of ACF There are a few steps to use ACF to find the recurring patterns in our studied datasets: Step 1) We first compute "daily average", which refers to the mean value of the target variable calculated for each individual day over the entire time period, and then apply the ACF to this historical daily average to identify the appropriate segment length (i.e., window size). If the dataset has only one value per day, we apply the ACF on the data itself. Step 2) We analyze a varying range of lags ranging from 1 to half of the length of the dataset. We choose half of the length of the dataset to cover enough data to detect most of the possible seasonality lengths and leave enough data as the test data. Step 3) We go through the correlation values and find local maxima, which are points where the autocorrelation forms the "peak". We sort the autocorrelation peaks decrementally based on their values. The first peak lag (e.g. lag l) with the highest correlation value represents seasonality in every l step. Step 4) We apply the following filter to pick the most suitable lag. From the sorted peak lags, we pick the first lag greater than or equal to 14 days. We set the minimum lag to 14 days to 1) leave enough data for model training and reduce the risk of overfitting and high variance, and 2) reduce the risk of capturing noise and anomalies rather than a stable recurrent pattern (Hyndman and Athanasopoulos 2018).

Forecasting In our proposed approach, we compare the similarities between a new time window's data with previous time windows to identify reoccurring patterns. However, in practical scenarios, the entire data in the new time window may not be available (i.e., unseen) when we make predictions for instances in the window. For instance, if it is the beginning of February, we do not have the distribution pattern for the entire month of February. Therefore, we need to use a forecasting method to forecast the distribution for February.

We use the following forecasting approaches to approximate the distribution of the new unseen data in production. The approach selection reason is explained in detail in the approach description.

- *Approach 1 - Exponential Smoothing (ES)*: ES is a forecasting algorithm for the time series dataset that uses weighted averages of past observations to forecast future values, where weights diminish exponentially as the data ages. This algorithm assigns greater importance to recent observations while exponentially decreasing the importance of older data points (Simplilearn 2023). This technique is suitable for datasets with seasonal patterns.
- *Approach 2 - Similarity Assumption (SA)*: Mallick et al. (2022) mentioned that models trained on training data with distribution A can be accurate for any test data with distribution B where B is similar to A. Similar to the idea presented in prior work (Brzezinski and Stefanowski 2013; Tahmasbi et al. 2020), we assume that the data distribution in the upcoming window W will follow the data distribution from the past window. Throughout this paper, the output of forecasting methods is called forecasted data.

Compute Similarity We use two similarity measurement methods: 1) the Wasserstein distance (WD) and 2) total variation distance (TVD) to compute the similarity of each data window to its preceding windows. Since we do not know the distribution of the immediate future data where we will use the time series models, we use these similarity measures to compare the forecasted distributions in the immediate-future window with those of previous data windows. The forecasted data are produced by forecasting approaches 1 (ES) and 2 (SA). Then we compute the ratio of similar segments to their non-adjacent previous data segments. For computing the similarity ratio, we use (7) presented below.

We decided to run our experiments with two different similarity measurement metrics to examine the impact that a choice of similarity measurement method has on the results of our study. Both similarity measurement techniques are applied on the ES and SA forecasting results. The description of the two methods is provided below.

- *Wasserstein distance (WD)*: Wasserstein distance (WD) or Earth Mover's Distance (EMD) computes the distance between two probability distributions over a given metric space. WD quantifies the least amount of effort needed to change one distribution into another. The amount of effort is measured by the distribution mass that needs to be moved times the mean distance that has to be moved (Wikipedia Contributors 2024d). The reason for choosing WD is that it concerns where the probability mass is located and how far it must be moved (AmazonWebServices 2024b). Probability mass means

the amount of probability assigned to specific locations in the space under the distribution. As seen in (5), $W_p(\mu, \nu)$ distance computes the minimum "cost" of transporting mass between the distributions μ and ν. In this equation, μ and ν are probability distributions on a space $\mathcal{X}$, $d(x, y)$ is a distance function that computes how far x is from y, and p is a positive real number that depicts the type of Wasserstein distance. The set $\Pi(\mu, \nu)$ contains all couplings (or joint distributions) whose marginals are μ and ν, and the expression $\inf_{\pi \in \Pi(\mu,\nu)}$ denotes that we choose the coupling π that minimizes the integral of $d(x, y)^p$ with respect to π. In the end, the integral $\int_{\mathcal{X}\times\mathcal{X}} d(x, y)^p \, d\pi(x, y)$ sums (integrates) the cost of moving one unit of mass from x to y. Raising this integral to the power of $1/p$ completes the description of the Wasserstein distance (Aubinsps-Frankowski 2024).

$$W_p(\mu, \nu) = \left(\inf_{\pi \in \Pi(\mu,\nu)} \int_{\mathcal{X}\times\mathcal{X}} d(x, y)^p \, d\pi(x, y) \right)^{\frac{1}{p}} \tag{5}$$

– *Total variation distance (TVD)*: The total variation distance (TVD) measures the difference between two probability distributions. Let P and Q be two probability distributions over the same sample space. $P(x)$ and $Q(x)$ are the probability of the event x happening based on the distributions P and Q, respectively (Reddit Contributors 2024). In this case, TVD is the maximum difference in probabilities that these distributions assign to the same events (Levin and Peres 2017; Wikipedia Contributors 2024c). Interpretation of TVD is straightforward, making it a suitable choice for our study.

$$d_{\mathrm{TV}}(P, Q) = \frac{1}{2} \sum_{x} |P(x) - Q(x)| \tag{6}$$

$$\text{Similarity ratio} = \frac{\text{Number of similar non adjacent segments}}{\text{Total number of segments}} \tag{7}$$

Analysis Method The steps we follow to do our preliminary study are: The process begins by normalizing the dataset columns. Next, the daily average of the target variable is calculated. Then, ACF is applied to check if seasonality exists in the datasets. If seasonality is detected, we will pick the segment length that is most proper. Once the proper segment length is established, forecasting methods are applied to predict the target values for the upcoming data segment.

Then, the distribution similarity between the forecasted segment of data and previous segments of data is computed using the WD and TVD metrics. This way, we identify the most similar preceding segment of data.

4.3 Preliminary Study Results

This section provides the results achieved from the preliminary study.

Table 3 ACF seasonality results

Dataset	Seasonality length (days)
NSW Electricity	14
Zurich Electricity	14
Oil temperature	7
Weather	15
NIFTY-50	258

Table 4 Percentage of the most similar non-adjacent data segments ratio

Dataset	SA - Wass	SA - TVD	ES - Wass	ES - TVD
NSW Electricity	74%	**82%**	82%	63%
Zurich Electricity	46%	**69%**	50%	61%
Oil temperature	79%	**94%**	86%	67%
Weather	58%	**70%**	62%	50%
NIFTY50	41%	**66%**	50%	58%

In the names of columns two to five, A - B means the forecasting method is A and the similarity measurement method is B

4.3.1 Finding the Proper Window Size

We present the result achieved by ACF in this section. After applying ACF, we realized that there are seasonalities in the studied datasets. Table 3 shows the seasonal length found for each dataset. The NSW Electricity and the Zurich Electricity datasets have biweekly seasonal patterns. The Oil temperature dataset has a weekly seasonal pattern. In other words, we found weekly or biweekly seasonal patterns in 60% of our studied datasets. The Weather dataset and NIFTY-50 datasets have 15 days and 258 days of seasonal patterns, respectively.

In the next step, we divide the datasets into segments with the achieved seasonality length and then compute the similarity between segments as described below.

Similarity Measurement We measured the similarity between each segment with two similarity measurement approaches of WD and TVD. Table 4 shows the ratio of the most similar segments for each forecasting method for each dataset. Similar segments have a similar data distributions.

In this study, we measured the similarity between each data segment and its preceding data segments. Then, for each dataset, we measured the ratio of the non-adjacent data segments that are the most similar data segments. As Table 4 shows, there are high similarities (more than 41% for all datasets) between the segments and their non-adjacent preceding segments. Oil temperature has the highest similarity among its segments. NIFTY-50 shows the lowest similarity among its segments. In 70% of the cases, the TVD similarity measurement method achieved a higher ratio over the Wass method. Results achieved in Table 4 show that the SA forecasting method mixed with the TVD similarity measurement method achieves the highest similarity ratio for all datasets.

Summary of preliminary study results

- There are considerable re-occurrences of similar data distributions over the time span of our studied time series datasets.
- We found weekly or biweekly seasonal patterns in 60% of our studied datasets.
- All of the studied datasets show more than 41% similarities between the segments and their preceding segments, indicating the feasibility of our approach that leverages the similarity between anticipated distributions and previous distributions to reuse previously trained models.

5 Proposed Solution (Model Reuse Approach)

In the preliminary study, we found seasonal and reoccurring distribution patterns in time series datasets. Therefore, in this section, we aim to reuse our previously trained models in the future for similar data distributions. We propose our model maintenance approach, which is model-agnostic and requires less maintenance time and cost, to help MLOps engineers and companies save resources and time.

First, we propose our modified MLOps pipeline and its components and explain how it works in the production environment. Then, we explain and implement our model reuse approach in our SimReuse tool.

5.1 Proposed MLOps Pipeline

The proposed MLOps pipeline is shown in Fig. 4. Some components of our MLOps pipeline and their description below are adapted from existing work (Majidi et al. 2022; Amershi et al. 2019; Hepworth 2022; Google Cloud 2024). The subscript "New" identifies the components that we add to the existing MLOps pipeline. Each new component is explained in more detail in what follows:

Distribution Analysis$_{\text{New}}$ To determine whether the dataset exhibits seasonality or reocurring patterns, we apply the ACF on the daily average of the target variable values of the training data (historical data). The ACF helps identify the presence of such patterns and, if they exist, reveals the reocurrence interval (i.e., the window length) of the repeating distribution. Then we follow the approach explained in Section 4.2 to identify the proper segment length to use in the rest of the pipeline.

Model Registry Like the MLOps pipeline, we store the trained models in the model registry.

Forecasting$_{\text{New}}$ Our tool aims to work in real-time systems. Since we do not have the data distribution for future data, we need approaches to forecast the target values in the next window. We use two approaches, SA and ES, to forecast the target values in the next window.

Compute Similarity$_{\text{New}}$ One essential contribution of this research is reducing the frequency of model training by reusing existing models. To achieve this, we propose utilizing the same model for several windows with data distributions similar to the train data. The initial step

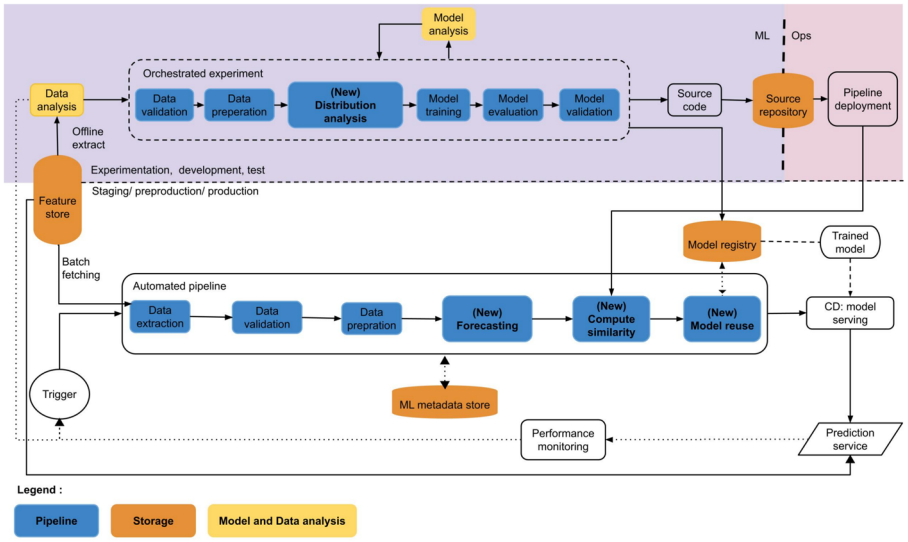


Fig. 4 Proposed improved MLOps pipeline

involves calculating the distribution similarity score between each window and its preceding windows. We use two methods, including 1) Wasserstein distance and 2) total variation distance, to compute the distribution similarity scores. We apply the distribution similarity measurement methods on the daily average of the target values.

After computing the distribution similarity score between each window and its preceding windows, we identify the most similar window. This window is the one among the previous windows that is most similar to the inference data. The most similar window can be the previous window or one of the earlier windows.

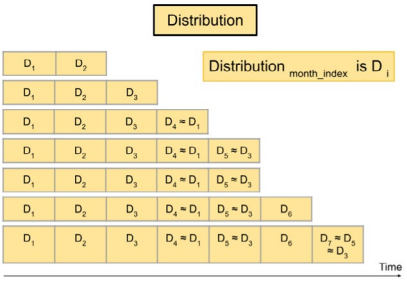
Model Reuse$_{\text{New}}$ Training a machine learning model is costly and time-consuming. Therefore, in the model reuse component, we aim to reuse a model trained on a similar distribution to our current inference data instead of training a new model.

A new model is trained on the most recent data segment if it is the most similar data segment. Otherwise, we reuse the model trained on the most similar data segment. In other words, models trained on data segments with distribution "X" are reused for data segments with distribution "Y", provided that Y is similar to X, as determined by the similarity scores from the similarity measurement step. This model reuse component is explained in detail in section 5.2.

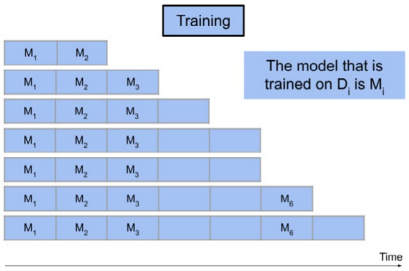
5.2 Model Reuse

In this section, we explain our model reuse approach. There is a "model reuse" component in the proposed MLOps pipeline in Fig. 4. Figure 5 shows an example of the model reuse approach. Also, Fig. 6 breaks down this component using a flowchart to explain

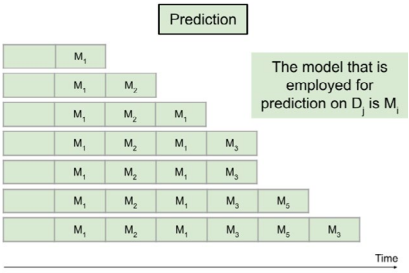
it in more detail. In the model reuse approach, we measured the distribution similarity between each data (daily average of the target variable values) segment and its preceding data segments using WD and TVD distribution similarity measurement metrics and selected the most similar data segment (data distribution) with the lowest similarity measurement metric value. For instance, for the data (target variable values) of segment #10, we measure its distribution similarity with its preceding data segments. For example, If data segment #5 yields the lowest Wasserstein Distance (WD) and Total Variation Distance (TVD), indicating the highest similarity, then segment #5 is selected for the remainder of the study.



(a) An example of data distribution in each time window



(b) Continuing the example: Models that are trained on data of each time window



(c) Continuing the example: The model used for prediction in each time window

Fig. 5 An example of the model reuse approach

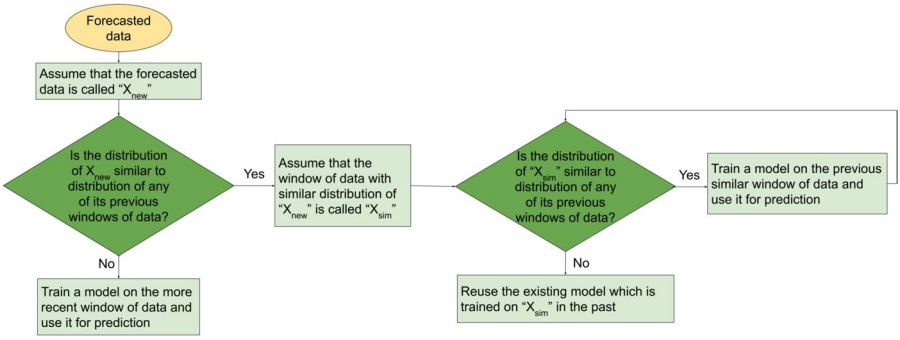


Fig. 6 Model reuse flowchart

Algorithm 1 Model Reuse Algorithm.

Input: dataset, window_size, target_column, windows_similarity_dictionary, forecasting_approach
Initialize: saved_models ← [ ] ▷ store the models in a list
for i ← window_size **to** len(dataset) **step** window_size **do** ▷ loop through the dataset in windows
  train ← dataset[$i -$ window_size : i] ▷ assign train data
  test ← dataset[i : $i +$ window_size] ▷ assign test data
  y_{train} ← train[target_column]
  X_{train} ← train.drop(columns=[target_column])
  y_{test} ← test[target_column]
  X_{test} ← test.drop(columns=[target_column])
  if len(test) < window_size **then**
    break ▷ break if the test size is less than window_size
  end if
  if forecasting_approach == "ES" **then**
    window_index ← round(i/window_size) $-$ 1 ▷ calculate window index for ES approach
  else if forecasting_approach == "SA" **then**
    window_index ← round(i/window_size) ▷ calculate window index for SA approach
  end if
  if window_index ∈ windows_similarity_dictionary **then** ▷ check if window index is in similarity dictionary
    similar_window_index ← windows_similarity_dictionary[window_index] ▷ find similar window index
    if similar_window_index ∈ windows_similarity_dictionary **then** ▷ check if there is a chain in similarities
      if forecasting_approach == "ES" **then**
        previous_model_i ← (window_index + 1) × window_size
      else if forecasting_approach == "SA" **then**
        previous_model_i ← window_index × window_size
      end if
      test2 ← dataset[previous_model_i : previous_model_i + window_size]
      y_{test2} ← test2[target_column]
      X_{test2} ← test2.drop(columns=[target_column])
      model ← train_model(X_{test2}, y_{test2}) ▷ train model M on the previous test data
      saved_models[similar_window_index] ← model ▷ replace the stored model with the new one
      append model to saved_models ▷ save the new model
      y_{pred} ← model.predict(X_{test})
      MSE ← mean_squared_error(y_{test}, y_{pred}) ▷ calculate MSE
      MAE ← mean_absolute_error(y_{test}, y_{pred}) ▷ calculate MAE
    else
      model ← saved_models[similar_window_index] ▷ retrieve the existing model
      append model to saved_models ▷ save the reused model
      y_{pred} ← model.predict(X_{test})
      MSE ← mean_squared_error(y_{test}, y_{pred}) ▷ calculate MSE
      MAE ← mean_absolute_error(y_{test}, y_{pred}) ▷ calculate MAE
    end if
  else
    model ← train_model(X_{train}, y_{train}) ▷ train a new model on the most recent window
    y_{pred} ← model.predict(X_{test})
    MSE ← mean_squared_error(y_{test}, y_{pred}) ▷ calculate MSE
    MAE ← mean_absolute_error(y_{test}, y_{pred}) ▷ calculate MAE
    append model to saved_models ▷ save the new model
  end if
end for

The Algorithm 1 explains the methodology for model reuse step by step. The step-by-step explanation of the Algorithm 1 is given below:

- (Line 1) The inputs of the algorithm are 1) the *dataset*, 2) the size of the window (*window-size* "W"), 3) the *targetcolumn*, 4) a *windows_similarity_dictionary* where the keys are windows indices, and the values are the corresponding similar windows to the keys, and 5) *forecastingapproach*.
- (Line 2) We store all the models in a list called *saved_models*.
- (Line 3 to line 49) We go through the *dataset* in a for loop.
- (Line 4 to line 12) *train* and *test* sets are assigned. We exit the for loop if the test data length is shorter than the *windowsize*.
- (Line 13 to line 17) We assign the *window_index* based on the *forecastingapproach*.
- (Line 18 to line 19) If the *window_index* is among the values in the *windows_similarity_dictionary*, assign the key to the *similar_window_index*.
- (Line 20 to line 30) If the *similar_window_index* is among the values in the *windows_similarity_dictionary*, it means that we have a chain of similarities. If we have a chain of similarities, we train a model (B) on the data of the *similar_window_index*.
- (Line 31 to line 32) We store the *model* and use it for prediction on the test data.
- (Line 33 to line 34) We measure the mean squared error and mean absolute error.
- (Line 35 to line 41) If we do not have a chain of similarities, we simply get the saved model and use it to make the prediction. Then, we measure the mean squared error and mean absolute error.
- (Line 42 to line 49) If the *similar_window_index* is not among the values in the *windows_similarity_dictionary*, we train a model on the most recent window of data and use it to make predictions. Then, we measure the mean squared error and mean absolute error. Finally we store the *model* for further reuse in the future.

It is worth mentioning that Lyuet al. (2021a) mentioned that the most frequently used model maintenance and concept drift adaptation method is model retraining. However, the retraining model takes time and is costly. Some companies with large models and massive datasets may be unable to afford it. Besides, the Stationary model's performance may degrade over time. Thus, our model reuse preserves the model performance while keeping the maintenance time and cost low.

6 Evaluation

We proposed our approach in Section 5. In this section, we evaluate the effectiveness of our method. In this section, we first explain our analysis approach; then, we present the results that we have achieved.

6.1 Evaluation Context

This section explains the steps we took to evaluate our approach.

Baseline Selection Process To evaluate the effectiveness of our approach, we aim to compare our approach with state-of-art baselines. To find the baselines, we follow the steps below:

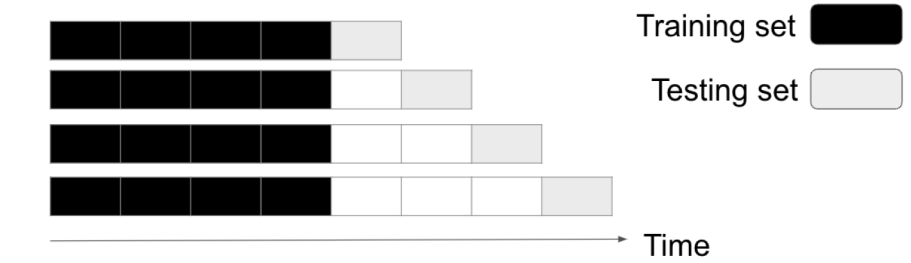
- We search for related papers on Google Scholar and Engineering Village (Compendex, Inspect databases).
- We search for the following query to find the related papers: ("Software engineering" OR aiops OR mlops) AND ((data OR concept OR model) AND (drift OR shift)) AND (mitigat* OR retrain OR maintenance OR maintain* OR detect OR detection). We aim to include all keywords and concepts related to model maintenance and MLOps that might be mentioned in other research works (e.g. AIOps).
- We filter the papers based on their publication year, discarding those published before 2018.
- To select the baselines, we follow the criteria mentioned below:
  - Our approach is model-agnostic, which is compatible with all models and time series datasets. Consequently, one of our baseline selection criteria is its model-agnostic nature.
  - We discard the approaches that are made for specific algorithms, systems, purposes, or use cases.
  - The baseline approaches must have an immediate inference time because time is important in some applications (e.g. AIOps applications) (Lyuet al. 2024).
  - The baseline approaches must have been used in other research papers.

After following the steps above, we identify three baseline approaches that are widely used in the related research papers (Lyuet al. 2021a; Poenaru-Olaru et al. 2024). The baseline approaches are 1) the stationary model, 2) the periodically updated model, and 3) the informed model retraining. The description of the baseline methods and their training process is provided below. Also, we include a random prediction model as our last baseline.

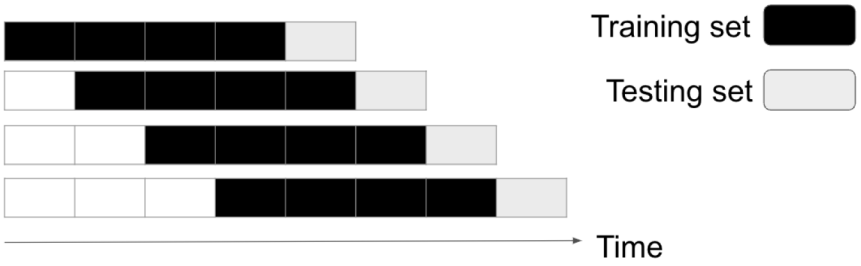
1) Periodically Updated Model In the periodically updated model approach, a model is trained on the most recent time window of data (e.g. last month) and tested on the next time window of data (test data). Note that the sliding window of the training data and test data move forward over time. Besides, we do not use all the historical data to update the model. Instead, we use only the latest sliding window. This is because using all the data is hard to scale, especially when the model needs frequent updates. Moreover, using all the past data may not necessarily improve the model's performance as much as using just the new data. Figure 7b shows the periodically updated model approach (Lyuet al. 2021a).

We provide examples of the prediction phase of periodically updated in Fig. 9. Figure 9 shows how the periodically updated model makes predictions in each time window. In Fig. 9, each box is a time window, and M_k is the model trained on the time window "k".

Periodically Updated Model Training Process During the model retraining process, we employ four distinct algorithms, RF, XGB, LSTM, and GRU to minimize the impact of algorithm choice on the results. We periodically update the models with a wide range of



(a) Stationary model approach (?)



(b) Periodically updated model approach (?)



(c) Informed retraining approach (?)

Fig. 7 Stationary model approach (Lyuet al. 2021a). Periodically updated model approach (Lyuet al. 2021a). Informed retraining approach (Poenaru-Olaru et al. 2024)

segment lengths, including five days, 15 days, 31 days, 45 days, 60 days, 75 days, and 90 days for the NSW Electricity, Zurich Electricity, Oil temperature, Weather datasets, and store their MSE and MAE. For the NIFTY-50 dataset, we consider bigger segment lengths because we have more years of data available. The segment lengths we used for periodically retraining the NIFTY-50 dataset are 50 days, 150 days, 310 days, 450 days, 600 days, 750 days, and 900 days. Having a wide range of segment lengths allows us to have full coverage of different possible segment lengths. Then, we periodically update the model with the segment length achieved the least MSE.

2) Stationary Model In the Stationary model approach, a model is trained on a first time window (e.g., the first month) and never gets updated. Figure 7a

We provide examples of the prediction phase of the stationary models in Figs. 9 and 8. Figure 8 depicts the stationary model's prediction phase, indicating the model used for predictions in each time window. In Fig. 8, M_1 is the model trained on the training data.

Stationary Model Training Process We use four different algorithms, RF, XGB, LSTM, and GRU to lessen the impact of training methods on our results. Then, we train a stationary model on the selected segment length by the model reuse approach.

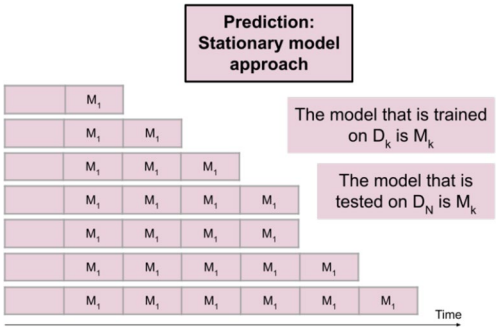


Fig. 8 An example of the prediction phase of the stationary model approach. D_K is the distribution of segment K. M_K is the model that is trained on D_K

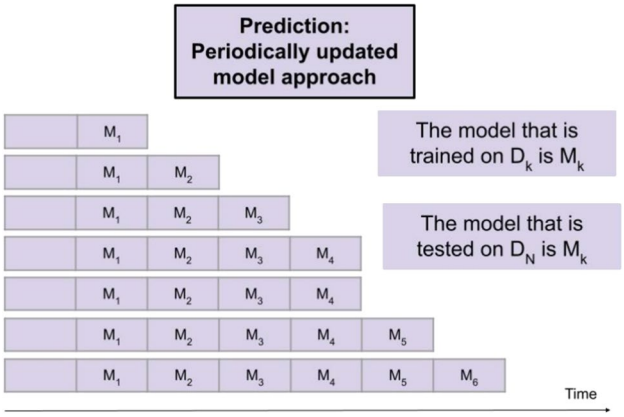


Fig. 9 An example of the prediction phase of the periodically updated model approach. D_K is the distribution of segment K. M_K is the model that is trained on D_K

3) Random Baseline We generate random predictions within the range of minimum and maximum values of the inference data's target column. Then, we evaluate the predicted result.

4) Informed retraining (Poenaru-Olaru et al. 2024) In the informed retraining approach, the model is retrained whenever drift is detected in the recent test data segment (Poenaru-Olaru et al. 2024). To detect the drift, we followed the approach used by Lyu et al. (2021a). Following the Poenaru-Olaru et al. 2024 approach, concept drift is detected by training a model on one time period and testing it on the next, then using a two-proportion Z-test to compare the model's prediction error rates between the training and testing periods. If the statistical test shows a significant difference (p-value $<$ 0.05), it indicates that concept drift has occurred. If the drift is detected for a segment of test data, the model is retrained on the previous segment of data. If no drift is detected, the last trained model is used until a drift occurs in a new test data segment. Figure 7c shows the informed model retraining approach. As shown in Fig. 7c, drift is detected in the third test data segment. Thus, a model retraining message is sent to update the model. Also, Fig. 10 shows an example of the prediction phase of the informed retraining approach. Additionally, we measure the ratio of drifted segments of data based on the (8).

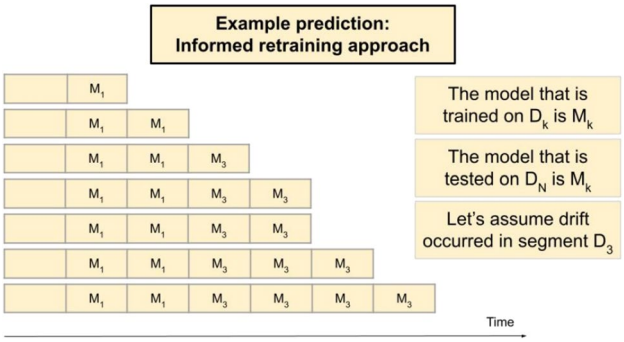


Fig. 10 An example of the prediction phase of the informed retraining approach. D_K is the distribution of segment K. M_K is the model that is trained on D_K

$$\text{Drift ratio} = \frac{\text{Number of concept drifted segments}}{\text{Total number of segments}} \tag{8}$$

Informed Retraining Process We first detect the drifts using the method mentioned in Lyuet al. (2021a). Secondly, we retrain the model on the last segment of the data if the drift occurs. During the model retraining process, we employ four distinct algorithms, RF, XGB, LSTM, and GRU, to minimize the impact of algorithm choice on the results.

Table 5 shows the ratio of concept-drifted segments. As seen in Table 5, all of our studied datasets have drifted segments. The Oil temperature has the lowest drifted segment ratio with 31% drift, and the Zurich electricity dataset has the highest drifted segments with 76% drift.

6.2 RQ1: How does SimReuse Perform Regarding ML Performance Metrics?

We have proposed a new model maintenance approach and developed our SimReuse tool so far. In this section, we aim to evaluate the performance of our proposed method.

Analysis Method To evaluate SimReuse with the baselines, we follow the steps below: We developed and applied five distinct modeling approaches - periodically updated models, stationary models, informed retraining, random models, and model reuse strategies—across all five datasets. After implementing these approaches, we conducted a performance comparison to evaluate the effectiveness of each learning algorithm on different datasets.

Table 5 Percentage of concept drifted segments

Dataset	Percentage of concept drifted segments
NSW Electricity	50%
Zurich Electricity	76%
Oil temperature	31%
Weather	69%
NIFTY-50	36%

In the proposed solution in section 5, we develop SimReuse to reduce the number of unnecessary retrainings. In RQ1, we aim to evaluate its performance and compare it with the baselines.

Results Tables 6 and 7 show the MSE and MAE results of the model reuse approach using LSTM and RF as the learning algorithms in four different settings. They also compare the results of different baseline approaches. Please note that the performance results of the GRU and the XGB learning algorithms are presented in the appendix.

We run the model reuse approach for each ML learning algorithm for each dataset with four different configurations. These four configurations are different combinations of results from two similarity metrics (WD and TVD) and two forecasting methods (SA and ES). The reported results are only for the time windows for which we utilized the "model reuse" approach for their inference. The best two performances in each row are highlighted in bold text.

Table 7 represents the MSE and MAE results of the model reuse approach employing RF as the learning algorithm under four different configurations. They also compare the results of different baseline approaches.

As seen in the result Tables 6 and 7, the periodically model update approach achieves the lowest MSE and MAE values among the baselines and model reuse approach in most of the situations. However, there are cases highlighted by ** in the result tables where the model reuse approach achieves the lowest error compared to other baselines, even to the periodical retraining baseline. According to the results presented in Tables 6 and 7, 50%, 60%, 60% and 70% of the configurations, respectively, achieved either the lowest or second-lowest error using the model reuse approach after periodical retraining. Furthermore, the results of the Tables 6 and 7 show that the forecasting method and similarity metric do not have a significant effect on the MSE and MAE of the model reuse approach.

Comparison of the Results We use the Mann-Whitney U statistical test to compare the results of different approaches. We compare the results of the model reuse approach with all of the other approaches. Table 8 shows the comparison between the different approaches. To compare the results of different approaches, we concatenated the MSE and MAE results of different learning algorithms. Then we applied the Mann-Whitney U statistical test on the results. **As seen in** Table 8, **the comparison between the periodically updated model and the model reuse approach achieves the highest p-value greater than 0.05, which shows an insignificant difference between the results. Therefore, this result shows that we can use the model reuse approach instead of periodical retraining approaches.** This way, we can save time, cost, and resources. Furthermore, the comparison between the informed retraining and the model reuse approach gives a high p-value (above 0.05), meaning there is also no significant difference between their results. Moreover, stationary and random baselines have the smallest p-value (much smaller than 0.05) when compared with the model reuse approach. This result shows a greater difference between these two baselines and the model reuse approach.

Table 6 Performance comparison of different model update approaches (learning algorithm: LSTM)

(a) Performance measurement metric: MSE

Dataset	Similarity metric	Forecast approach	Stationary model MSE	Periodically updated model MSE	Model reuse MSE	Informed retraining MSE	Random model MSE
NSW Electricity	WD	SA	19×10^{-4}	$\mathbf{14\times10^{-4}}$	$\mathbf{12\times10^{-4}}$	19×10^{-4}	34×10^{-3}
NSW Electricity	TVD	SA	19×10^{-4}	$\mathbf{13\times10^{-4}}$	$\mathbf{11\times10^{-4}}$	20×10^{-4}	34×10^{-3}
NSW Electricity	WD	ES	19×10^{-4}	$\mathbf{14\times10^{-4}}$	$\mathbf{11\times10^{-4}}$	17×10^{-4}	34×10^{-3}
NSW Electricity	TVD	ES	19×10^{-4}	$\mathbf{15\times10^{-4}}$	$\mathbf{14\times10^{-4}}$	15×10^{-4}	34×10^{-3}
Zurich Electricity	WD	SA	13×10^{-3}	$\mathbf{11\times10^{-3}}$	$\mathbf{12\times10^{-3}}$	14×10^{-3}	59×10^{-3}
Zurich Electricity	TVD	SA	13×10^{-3}	$\mathbf{11\times10^{-3}}$	19×10^{-3}	$\mathbf{11\times10^{-3}}$	59×10^{-3}
Zurich Electricity	WD	ES	13×10^{-3}	$\mathbf{11\times10^{-3}}$	13×10^{-3}	$\mathbf{11\times10^{-3}}$	59×10^{-3}
Zurich Electricity	TVD	ES	13×10^{-3}	$\mathbf{12\times10^{-3}}$	14×10^{-3}	$\mathbf{10\times10^{-3}}$	59×10^{-3}
Oil temperature **	WD	SA	57×10^{-3}	$\mathbf{63\times10^{-4}}$	$\mathbf{51\times10^{-4}}$	91×10^{-4}	10×10^{-3}
Oil temperature	TVD	SA	57×10^{-3}	$\mathbf{62\times10^{-4}}$	31×10^{-3}	$\mathbf{84\times10^{-4}}$	10×10^{-3}
Oil temperature **	WD	ES	57×10^{-3}	$\mathbf{63\times10^{-3}}$	$\mathbf{46\times10^{-4}}$	10×10^{-3}	10×10^{-3}
Oil temperature	TVD	ES	57×10^{-3}	$\mathbf{63\times10^{-4}}$	$\mathbf{10\times10^{-3}}$	11×10^{-3}	10×10^{-3}
Weather	WD	SA	12×10^{-4}	$\mathbf{16\times10^{-5}}$	$\mathbf{17\times10^{-5}}$	22×10^{-5}	37×10^{-3}
Weather	TVD	SA	12×10^{-4}	$\mathbf{17\times10^{-5}}$	31×10^{-5}	$\mathbf{21\times10^{-5}}$	37×10^{-3}
Weather	WD	ES	12×10^{-4}	$\mathbf{15\times10^{-5}}$	45×10^{-5}	$\mathbf{19\times10^{-5}}$	37×10^{-3}
Weather	TVD	ES	12×10^{-4}	$\mathbf{16\times10^{-5}}$	47×10^{-5}	$\mathbf{18\times10^{-5}}$	37×10^{-3}
NIFTY-50	WD	SA	58×10^{-4}	$\mathbf{19\times10^{-4}}$	37×10^{-4}	$\mathbf{35\times10^{-4}}$	30×10^{-3}
NIFTY-50	TVD	SA	58×10^{-4}	$\mathbf{26\times10^{-4}}$	58×10^{-4}	$\mathbf{53\times10^{-4}}$	30×10^{-3}
NIFTY-50	WD	ES	58×10^{-4}	$\mathbf{20\times10^{-4}}$	67×10^{-4}	$\mathbf{52\times10^{-4}}$	30×10^{-3}
NIFTY-50 **	TVD	ES	58×10^{-4}	$\mathbf{24\times10^{-4}}$	$\mathbf{22\times10^{-4}}$	49×10^{-4}	30×10^{-3}

Table 6 (continued)

(b) Performance measurement metric: MAE

Dataset	Similarity	Forecast approach	Stationary model MSE	Periodically updated model MSE	Model reuse MSE	Informed retraining MSE	Random model MSE
NSW Electricity	WD	SA	29×10^{-3}	$\mathbf{16\times10^{-3}}$	$\mathbf{16\times10^{-3}}$	22×10^{-3}	10×10^{-2}
NSW Electricity	TVD	SA	29×10^{-3}	$\mathbf{16\times10^{-3}}$	$\mathbf{17\times10^{-3}}$	22×10^{-3}	10×10^{-2}
NSW Electricity	WD	ES	29×10^{-3}	$\mathbf{16\times10^{-3}}$	$\mathbf{16\times10^{-3}}$	19×10^{-3}	10×10^{-2}
NSW Electricity	TVD	ES	29×10^{-3}	$\mathbf{16\times10^{-3}}$	$\mathbf{19\times10^{-3}}$	$19\times\mathbf{10^{-3}}$	10×10^{-2}
Zurich Electricity	WD	SA	91×10^{-3}	$\mathbf{82\times10^{-3}}$	$\mathbf{85\times10^{-3}}$	91×10^{-3}	19×10^{-2}
Zurich Electricity	TVD	SA	91×10^{-3}	$\mathbf{83\times10^{-3}}$	10×10^{-2}	$\mathbf{83\times10^{-3}}$	19×10^{-2}
Zurich Electricity	WD	ES	91×10^{-3}	$\mathbf{82\times10^{-3}}$	88×10^{-3}	$\mathbf{83\times10^{-3}}$	19×10^{-2}
Zurich Electricity	TVD	ES	91×10^{-3}	$\mathbf{84\times10^{-3}}$	93×10^{-3}	$\mathbf{78\times10^{-3}}$	19×10^{-2}
Oil temperature	WD	SA	20×10^{-2}	$\mathbf{61\times10^{-3}}$	$\mathbf{54\times10^{-3}}$	72×10^{-3}	74×10^{-3}
Oil temperature	TVD	SA	20×10^{-2}	$\mathbf{63\times10^{-3}}$	11×10^{-2}	$\mathbf{72\times10^{-3}}$	80×10^{-3}
Oil temperature **	WD	ES	20×10^{-2}	$\mathbf{61\times10^{-3}}$	$\mathbf{51\times10^{-3}}$	80×10^{-3}	80×10^{-3}
Oil temperature	TVD	ES	20×10^{-2}	$\mathbf{62\times10^{-3}}$	$\mathbf{76\times10^{-3}}$	84×10^{-3}	80×10^{-3}
Weather	WD	SA	24×10^{-3}	$\mathbf{95\times10^{-4}}$	$\mathbf{96\times10^{-4}}$	11×10^{-3}	15×10^{-2}
Weather	TVD	SA	24×10^{-3}	$\mathbf{91\times10^{-4}}$	11×10^{-3}	$\mathbf{96\times10^{-4}}$	15×10^{-2}
Weather	WD	ES	24×10^{-3}	$\mathbf{91\times10^{-4}}$	13×10^{-3}	$\mathbf{99\times10^{-4}}$	15×10^{-2}
Weather	TVD	ES	24×10^{-3}	$\mathbf{94\times10^{-4}}$	13×10^{-3}	$\mathbf{10\times10^{-3}}$	15×10^{-2}
NIFTY-50	WD	SA	70×10^{-3}	$\mathbf{32\times10^{-3}}$	$\mathbf{40\times10^{-3}}$	44×10^{-3}	10×10^{-2}
NIFTY-50	TVD	SA	70×10^{-3}	$\mathbf{35\times10^{-3}}$	74×10^{-3}	$\mathbf{60\times10^{-3}}$	10×10^{-2}
NIFTY-50	WD	ES	70×10^{-3}	$\mathbf{33\times10^{-3}}$	$\mathbf{60\times10^{-3}}$	66×10^{-3}	10×10^{-2}
NIFTY-50	TVD	ES	70×10^{-3}	$\mathbf{36\times10^{-3}}$	$\mathbf{30\times10^{-3}}$	61×10^{-3}	10×10^{-2}

The first two lowest error values are highlighted in each row. ** sign highlights the rows where the model reuse error is lower than the periodical error

Table 7

(a) Performance measurement metric: MSE

Dataset	Similarity metric	Forecast approach	Stationary model MSE	Periodically updated model MSE	Model reuse MSE	Informed re-training MSE	Random model MSE
NSW Electricity **	WD	SA	17×10^{-4}	$\mathbf{97\times10^{-5}}$	$\mathbf{86\times10^{-5}}$	12×10^{-4}	34×10^{-3}
NSW Electricity **	TVD	SA	17×10^{-4}	$\mathbf{97\times10^{-5}}$	$\mathbf{51\times10^{-5}}$	16×10^{-4}	34×10^{-3}
NSW Electricity **	WD	ES	17×10^{-4}	$\mathbf{97\times10^{-5}}$	$\mathbf{38\times10^{-5}}$	93×10^{-5}	34×10^{-3}
NSW Electricity **	TVD	ES	17×10^{-4}	$\mathbf{97\times10^{-5}}$	$\mathbf{97\times10^{-5}}$	83×10^{-5}	34×10^{-3}
Zurich Electricity **	WD	SA	95×10^{-4}	$\mathbf{63\times10^{-4}}$	$\mathbf{58\times10^{-4}}$	94×10^{-4}	60×10^{-3}
Zurich Electricity	TVD	SA	95×10^{-4}	$\mathbf{63\times10^{-4}}$	77×10^{-4}	$\mathbf{74\times10^{-4}}$	60×10^{-3}
Zurich Electricity	WD	ES	95×10^{-4}	$\mathbf{63\times10^{-4}}$	70×10^{-4}	$\mathbf{67\times10^{-4}}$	60×10^{-3}
Zurich Electricity	TVD	ES	95×10^{-4}	$\mathbf{63\times10^{-4}}$	61×10^{-4}	$\mathbf{58\times10^{-4}}$	60×10^{-3}
Oil temperature **	WD	SA	12×10^{-2}	$\mathbf{55\times10^{-4}}$	$\mathbf{42\times10^{-4}}$	21×10^{-3}	14×10^{-3}
Oil temperature	TVD	SA	12×10^{-2}	$\mathbf{55\times10^{-4}}$	62×10^{-3}	$\mathbf{14\times10^{-3}}$	14×10^{-3}
Oil temperature **	WD	ES	12×10^{-2}	$\mathbf{55\times10^{-4}}$	$\mathbf{53\times10^{-4}}$	10×10^{-3}	14×10^{-3}
Oil temperature	TVD	ES	12×10^{-2}	$\mathbf{55\times10^{-4}}$	82×10^{-4}	$\mathbf{56\times10^{-4}}$	14×10^{-3}
Weather **	WD	SA	12×10^{-3}	$\mathbf{23\times10^{-5}}$	$\mathbf{12\times10^{-5}}$	71×10^{-5}	37×10^{-3}
Weather	TVD	SA	12×10^{-3}	$\mathbf{23\times10^{-5}}$	10×10^{-4}	$\mathbf{41\times10^{-5}}$	37×10^{-3}
Weather	WD	ES	12×10^{-3}	$\mathbf{23\times10^{-5}}$	21×10^{-4}	$\mathbf{50\times10^{-5}}$	37×10^{-3}
Weather	TVD	ES	12×10^{-3}	$\mathbf{23\times10^{-5}}$	28×10^{-4}	$\mathbf{33\times10^{-5}}$	37×10^{-3}
NIFTY-50	WD	SA	33×10^{-4}	$\mathbf{15\times10^{-4}}$	$\mathbf{20\times10^{-4}}$	26×10^{-4}	30×10^{-3}
NIFTY-50	TVD	SA	33×10^{-4}	$\mathbf{15\times10^{-4}}$	$\mathbf{33\times10^{-4}}$	62×10^{-4}	30×10^{-3}
NIFTY-50	WD	ES	33×10^{-4}	$\mathbf{15\times10^{-4}}$	$\mathbf{36\times10^{-4}}$	66×10^{-4}	30×10^{-3}
NIFTY-50	TVD	ES	33×10^{-4}	$\mathbf{15\times10^{-4}}$	$\mathbf{24\times10^{-4}}$	45×10^{-4}	30×10^{-3}

Table 7 (continued)

(b) Performance measurement metric: MAE

Dataset	Similarity	Forecast approach	Stationary model MAE	Periodically updated model MAE	Model reuse MAE	Informed re-training MAE	Random model MAE
NSW Electricity	WD	SA	25×10^{-3}	$\mathbf{88\times10^{-4}}$	$\mathbf{90\times10^{-4}}$	11×10^{-3}	10×10^{-2}
NSW Electricity	TVD	SA	25×10^{-3}	$\mathbf{88\times10^{-4}}$	$\mathbf{91\times10^{-4}}$	12×10^{-3}	10×10^{-2}
NSW Electricity **	WD	ES	25×10^{-3}	$\mathbf{88\times10^{-4}}$	$\mathbf{82\times10^{-4}}$	92×10^{-4}	10×10^{-2}
NSW Electricity	TVD	ES	25×10^{-3}	$\mathbf{88\times10^{-4}}$	10×10^{-3}	$\mathbf{86\times10^{-4}}$	10×10^{-2}
Zurich Electricity **	WD	SA	78×10^{-3}	$\mathbf{56\times10^{-3}}$	$\mathbf{47\times10^{-3}}$	72×10^{-3}	19×10^{-2}
Zurich Electricity	TVD	SA	78×10^{-3}	$\mathbf{56\times10^{-3}}$	$\mathbf{57\times10^{-3}}$	62×10^{-3}	19×10^{-2}
Zurich Electricity **	WD	ES	78×10^{-3}	$\mathbf{56\times10^{-3}}$	$\mathbf{56\times10^{-3}}$	58×10^{-3}	19×10^{-2}
Zurich Electricity **	TVD	ES	78×10^{-3}	$\mathbf{56\times10^{-3}}$	$\mathbf{47\times10^{-3}}$	55×10^{-3}	19×10^{-2}
Oil temperature **	WD	SA	31×10^{-2}	$\mathbf{58\times10^{-3}}$	$\mathbf{44\times10^{-3}}$	99×10^{-3}	94×10^{-3}
Oil temperature	TVD	SA	31×10^{-2}	$\mathbf{58\times10^{-3}}$	16×10^{-2}	$\mathbf{84\times10^{-3}}$	94×10^{-3}
Oil temperature **	WD	ES	31×10^{-2}	$\mathbf{58\times10^{-3}}$	$\mathbf{47\times10^{-3}}$	72×10^{-3}	94×10^{-3}
Oil temperature	TVD	ES	31×10^{-2}	$\mathbf{58\times10^{-3}}$	63×10^{-3}	$\mathbf{60\times10^{-3}}$	94×10^{-3}
Weather **	WD	SA	62×10^{-3}	$\mathbf{37\times10^{-4}}$	$\mathbf{20\times10^{-4}}$	78×10^{-4}	15×10^{-2}
Weather	TVD	SA	62×10^{-3}	$\mathbf{37\times10^{-4}}$	84×10^{-4}	$\mathbf{52\times10^{-4}}$	15×10^{-2}
Weather	WD	ES	62×10^{-3}	$\mathbf{37\times10^{-4}}$	14×10^{-3}	$\mathbf{59\times10^{-4}}$	15×10^{-2}
Weather	TVD	ES	62×10^{-3}	$\mathbf{37\times10^{-4}}$	10×10^{-3}	$\mathbf{48\times10^{-4}}$	15×10^{-2}
NIFTY-50	WD	SA	35×10^{-3}	$\mathbf{17\times10^{-3}}$	$\mathbf{23\times10^{-3}}$	30×10^{-3}	10×10^{-2}
NIFTY-50	TVD	SA	35×10^{-3}	$\mathbf{17\times10^{-3}}$	$\mathbf{36\times10^{-3}}$	57×10^{-3}	10×10^{-2}
NIFTY-50	WD	ES	35×10^{-3}	$\mathbf{17\times10^{-3}}$	$\mathbf{32\times10^{-3}}$	69×10^{-3}	10×10^{-2}
NIFTY-50	TVD	ES	35×10^{-3}	$\mathbf{17\times10^{-3}}$	$\mathbf{21\times10^{-3}}$	51×10^{-3}	10×10^{-2}

The first two lowest error values are highlighted in each row. ** sign highlights the rows where the model reuse error is lower than the periodical error

Table 8 Comparison of different approaches performance results with Mann-Whitney U

Compared approaches	p-value	Significant or insignificant
Reuse & Periodical	**0.945**	**insignificant**
Reuse & Informed	0.574	insignificant
Reuse & Stationary	3×10^{-6}	significant
Reuse & Random	6×10^{-30}	significant

Periodical means Periodically updated model approach. Reuse means model reuse approach. The model update approaches with the most insignificant error value differences are highlighted in the table

Summary of RQ1 results

- Model reuse achieves the lowest or second lowest error after periodical retraining in 60% of the tested configurations with different datasets.
- The difference between the model reuse approach and the periodic model updating approach is not statistically significant. Both approaches perform better than the stationary and random approaches (with statistically significant differences).
- We encourage developers and MLOps engineers to utilize the model reuse approach or SimReuse tool in their projects as they achieve comparable results as the periodically updated model approach while being able to reduce the frequency of model updating.

6.3 RQ2: How does SimReuse perform regarding maintenance costs?

We have analyzed SimReuse's performance so far. In RQ2, we aim to evaluate ML models' maintenance time and financial costs and compare them with the time and costs of the baseline approaches. Thus, in this section, we report the savings of the model reuse approach regarding computation time and computation costs. Time is reported in "second" in this section.

Analysis Method To evaluate our approach's computation time and financial cost versus the baseline approaches, we run all the approaches, measure their computation cost and time, and compare them at the end. The most resource-intensive computations involve several tasks across the periodically updated model, stationary model, and model reuse approaches.

In the periodically updated model, essential resource-intensive tasks include determining the segment length that achieves the minimum MSE and periodically updating the model with this segment length. In the stationary model approach, training and applying the model to the inference data are costly processes. Model reuse requires resources for forecasting with the Exponential Smoothing (ES) approach, computing similarity metrics, and applying the model reuse approach using both the SA and ES forecasting methods. Lastly, informed model updating requires time and resources to run drift detection code and retrain the model whenever the drift is detected.

It's worth mentioning that we have two methods for forecasting: ES and SA. Only the ES approach is costly and time-consuming. Choosing the proper approach depends on the

developer in the development phase, and we do not always have to use the ES approach when running the pipeline.
The Table 9 shows the number of reduced model reuse approach retrainings compared to periodical retraining approach

Results Table 9 presents the reduced number of retraining in the model reuse approach compared to the periodically update approach. As seen in Table 9, on average, the model reuse approach reduces the number of retrainings by 29, 9, 52, 10, and 4 times for the NSW Electricity, Zurich Electricity, Oil Temperature, Weather, and NIFTY-50 datasets, respectively. Besides, Table 9 shows the percentage of reduced retraining count in the model reuse approach compared to the periodically update approach. As shown in Table 9, the model reuse approach reduces the number of retrainings by 46%, 36%, 50%, 42%, and 39% for the NSW Electricity, Zurich Electricity, Oil Temperature, Weather, and NIFTY-50 datasets, respectively, compared to the periodic model update approach. These results show that we can save a considerable amount of time and cost for the model update approach using our previously trained models.

Table 10 presents the computation time and cost of model reuse and the baseline approaches. We rescaled all the computation time and cost results over one year of using each method. The two highest computation costs are highlighted in the Table 10. As seen in the Table 10, the periodical retraining approach has the highest computaion cost. The informed model

Table 9 Model reuse reduced number of retrainings compared to periodical retraining

Dataset	Similarity metric	Forecast approach	Count of reduced retrainings	Percentage of reduced retraining count
NSW Electricity	WD	SA	27 times	42%
NSW Electricity	TVD	SA	35 times	55%
NSW Electricity	WD	ES	28 times	44%
NSW Electricity	TVD	ES	29 times	46%
Zurich Electricity	WD	SA	7 times	26%
Zurich Electricity	TVD	SA	13 times	50%
Zurich Electricity	WD	ES	9 times	34%
Zurich Electricity	TVD	ES	9 times	34%
Oil temperature	WD	SA	50 times	48%
Oil temperature	TVD	SA	54 times	52%
Oil temperature	WD	ES	44 times	42%
Oil temperature	TVD	ES	63 times	61%
Weather	WD	SA	10 times	41%
Weather	TVD	SA	13 times	54%
Weather	WD	ES	9 times	37%
Weather	TVD	ES	9 times	37%
NIFTY-50	WD	SA	4 times	33%
NIFTY-50	TVD	SA	6 times	50%
NIFTY-50	WD	ES	4 times	33%
NIFTY-50	TVD	ES	5 times	41%

Table 10 Comparison of the computaion time and costs of different model update approaches

Dataset	Algorithm	Stationary		Periodical		Reuse		Informed	
		Time	Cost	Time	Cost	Time	Cost	Time	Cost
NSW Electricity	LSTM	3.7	118×10	8, 62.3	$\mathbf{27{,}507 \times 10^{-6}}$	1, 06.5	$\mathbf{3{,}397 \times 10^{-6}}$	65.7	$2,095 \times 10^{-6}$
Zurich Electricity	LSTM	10.6	338×10^{-6}	13, 41.9	$\mathbf{42{,}806 \times 10^{-6}}$	1, 43.6	$4,580 \times 10^{-6}$	1, 96.8	$\mathbf{6{,}277 \times 10^{-6}}$
Oil temperature	LSTM	2.6	82×10^{-6}	5, 81.4	$\mathbf{18{,}546 \times 10^{-6}}$	1, 91.4	$\mathbf{6{,}105 \times 10^{-6}}$	1, 12.2	$3,579 \times 10^{-6}$
Weather	LSTM	19.9	634×10^{-6}	21, 02.7	$\mathbf{67{,}076 \times 10^{-6}}$	1, 74.2	$5,556 \times 10^{-6}$	2, 26.1	$\mathbf{7{,}212 \times 10^{-6}}$
NIFTY-50	LSTM	0.4	12×10^{-6}	4, 64.3	$\mathbf{14{,}811 \times 10^{-6}}$	1.8	$\mathbf{57 \times 10^{-6}}$	1.5	47×10^{-6}
NSW Electricity	GRU	4.2	133×10^{-6}	7, 92.3	$\mathbf{25{,}274 \times 10^{-6}}$	1, 06	$\mathbf{3{,}381 \times 10^{-6}}$	1, 04.9	$3,346 \times 10^{-6}$
Zurich Electricity	GRU	11.9	379×10^{-6}	1, 037.4	$\mathbf{33{,}093 \times 10^{-6}}$	1, 44.9	$4,622 \times 10^{-6}$	1, 68.9	$\mathbf{5{,}387 \times 10^{-6}}$
Oil temperature	GRU	2.2	70×10^{-6}	99.8	$\mathbf{3{,}183 \times 10^{-6}}$	23	$\mathbf{733 \times 10^{-6}}$	12.4	$395 \times 10-6$
Weather	GRU	21.2	676×10^{-6}	17, 22.3	$\mathbf{54{,}941 \times 10^{-6}}$	1, 76.8	$5,639 \times 10^{-6}$	2, 39.4	$\mathbf{7{,}636 \times 10^{-6}}$
NIFTY-50	GRU	0.2	6.38×10^{-6}	3, 45.7	$\mathbf{11{,}027 \times 10^{-6}}$	1.5	$\mathbf{47 \times 10^{-6}}$	1.2	38×10^{-6}
NSW Electricity	RF	0.5	15×10^{-6}	39.7	$\mathbf{1{,}266 \times 10^{-6}}$	3.7	118×10^{-6}	9.5	$\mathbf{303 \times 10^{-6}}$
Zurich Electricity	RF	3.9	124×10^{-6}	1, 34.3	$\mathbf{4{,}284 \times 10^{-6}}$	11.9	379×10^{-6}	23.9	$\mathbf{762 \times 10^{-6}}$
Oil temperature	RF	~ 0	~ 0	28.3	$\mathbf{902 \times 10^{-6}}$	3.9	124×10^{-6}	9.2	$\mathbf{293 \times 10^{-6}}$
Weather	RF	1.3	41×10^{-6}	2, 67.3	$\mathbf{8{,}526 \times 10^{-6}}$	26.6	848×10^{-6}	47.8	$\mathbf{1{,}524 \times 10^{-6}}$
NIFTY-50	RF	~ 0	~ 0	9.7	$\mathbf{309 \times 10^{-6}}$	0.1	3×10^{-6}	0.2	$\mathbf{6 \times 10^{-6}}$
NSW Electricity	XGB	~ 0	~ 0	6.8	$\mathbf{216 \times 10^{-6}}$	1	31×10^{-6}	9.5	$\mathbf{303 \times 10^{-6}}$
Zurich Electricity	XGB	~ 0	~ 0	11.9	$\mathbf{379 \times 10^{-6}}$	2.6	82×10^{-6}	14.6	$\mathbf{465 \times 10^{-6}}$
Oil temperature	XGB	~ 0	~ 0	9.2	$\mathbf{293 \times 10^{-6}}$	3.9	124×10^{-6}	9.9	$\mathbf{315 \times 10^{-6}}$
Weather	XGB	~ 0	~ 0	11.9	$\mathbf{379 \times 10^{-6}}$	1.3	41×10^{-6}	26.6	$\mathbf{848 \times 10^{-6}}$
NIFTY-50	XGB	~ 0	~ 0	0.3	$\mathbf{9 \times 10^{-6}}$	0.1	3×10^{-6}	0.2	$\mathbf{6 \times 10^{-6}}$

The first two highest costs in each row are highlighted in the table. Stationary, periodical, reuse, and informed columns are different studies model update approaches. The time and cost units are seconds and US dollars, respectively

update has the second-highest computation cost when using RF and XGB learning algorithms. It also has the second-highest computation cost for the Zurich Electricity and the Weather datasets when using LSTM and GRU learning algorithms. These results show that the model reuse approach costs less than the periodical retraining approach using different configurations for all the datasets. It also costs less than the informed retraining for all the datasets when using the RF and XGB learning algorithms. It costs less than informed retraining for the Zurich Electricity and the Weather datasets when using LSTM and GRU learning algorithms.

Table 11 shows the average of the computaion time and costs of different model update approaches. Based on the results of Table 11, model reuse approach saves \$11834, \$17709, \$3960, \$29709, and \$6515 (all costs should be multiplied by 10^{-6}) per year compared to the periodical retraining approach. Besides, based on the results of Table 11, the model reuse approach saves \$791, \$1284 per year for updating the model compared to the informed retraining approach for the Zurich Electricity and the Weather datasets (all costs should be multiplied by 10^{-6}). Another interesting finding is that the informed model retraining approach costs more than the model reuse approach on average based on the results of Table 11. Furthermore, **the periodical and informed model retraining approaches take 8.7 and 1.13 times more computation time and cost than the model reuse approach, respectively.**

In some companies, training a model can take a long time (e.g., a few days or weeks) due to the complexity of the algorithm and the large volume of data. The longer the training takes, the more expensive it becomes. For example, imagine a scenario where training a model on ml.c5.24xlarge AWS SageMaker instance takes a week. In this case, the cost of periodically retraining the model will be \$822 each time, which is expensive. Therefore, we believe that our model reuse approach is beneficial for companies to save time and financial resources by reducing the number of unnecessary training. In addition, periodic and informed retraining come with extra costs that are not easy to measure. For example, training, testing, deploying new models, and detecting drift take up employees' time, which can become expensive over time.

It is worth mentioning that the model reuse approach will perform as well as a periodically updated model in the worst-case scenario. The worst-case scenario is where there is no similar distribution in the past time series dataset. In that case, the model reuse approach can not find a similar distribution in the past data, so it will retrain a new model on the most recent data. However, even in this scenario, the computation cost will not exceed the periodical retraining because forecasting and similarity measurement computation cost less than finding the proper segment length for the periodically updating model. Therefore, the model reuse approach is more suitable for the datasets with seasonality and recurrent patterns in the distribution.

It is essential to point out that storage of the model is costly. If we store 1GB of data on an AWS S3 bucket, it will cost \$0.023 per month (AmazonWebServices 2025). The size of our biggest model is an RF model for the Weather dataset that takes 12MB of space on average. If we follow the model reuse approach, in the worst-case scenario where there is no similarity between each segment and its preceding segments, we should store 24 RF models per year, each of which takes 12MB on average. Thus, in total, the maximum required storage in this scenario would be 24×12MB, which is equal to a maximum of 288 MB at the end of

Table 11 Comparison of the average of the computation time and costs of different model update approaches for each dataset

Dataset	Stationary		Periodical		Reuse		Informed	
	Time	Cost	Time	Cost	Time	Cost	Time	Cost
NSW Electricity	2.1	66×10^{-6}	4, 25.2	$\mathbf{13{,}565\times10^{-6}}$	54.3	$\mathbf{1{,}731\times10^{-6}}$	47.4	$1,511\times10^{-6}$
Zurich Electricity	6.6	210×10^{-6}	6, 31.3	$\mathbf{20{,}140\times10^{-6}}$	75.7	$2,431\times10^{-6}$	1, 09.0	$\mathbf{3{,}222\times10^{-6}}$
Oil temperature	1.2	38×10^{-6}	1, 79.6	$\mathbf{5{,}731\times10^{-6}}$	55.5	$\mathbf{1{,}771\times10^{-6}}$	41.3	$1,145\times10^{-6}$
Weather	10.6	337×10^{-6}	10, 26	$\mathbf{32{,}730\times10^{-6}}$	94.7	$3,021\times10^{-6}$	1, 34.9	$\mathbf{4{,}305\times10^{-6}}$
NIFTY-50	0.15	4×10^{-6}	2, 05	$\mathbf{6{,}539\times10^{-6}}$	0.8	$\mathbf{27\times10^{-6}}$	0.7	24×10^{-6}
Average	4.1	131×10^{-6}	4, 93.4	$\mathbf{15{,}741\times10^{-6}}$	56.2	$1,796\times10^{-6}$	66.6	$\mathbf{2{,}041\times10^{-6}}$

The first two highest costs in each row are highlighted in the table. Stationary, periodical, reuse, and informed columns are different studied model update approaches. The time and cost units are seconds and US dollars, respectively

one year. 228MB costs $0.077 per year. Our approach needs the storage of historical models while the baseline approaches do not store the history of models.

Summary of RQ2 results

- Periodical retraining is the most costly model maintenance approach.
- The model reuse approach reduces the cost and time of model maintenance compared to periodic and informed retraining, while maintaining the performance comparable to both approaches.
- On average, the periodical and informed model retraining approaches take 8.7 and 1.13 times more time and cost than the model reuse approach, respectively.

7 Discussion and Implication

This section further discusses the implications of our novel MLOps pipeline, model reuse approach, and SimReuse tool for companies, researchers, cloud service providers, and MLOps engineers.

Companies and Organizations Our RQ1 and RQ2 results show that the proposed pipeline and the SimReuse tool will help the companies reduce the number of unnecessary retraining, which leads to saved time, cost, and effort. Therefore, organizations, startups and companies can reduce the budget and time spent on retraining their models and invest time and money in more beneficial areas.

Lyu et al. (2021a) mentioned that periodical model retraining is the most frequently used model maintenance and concept drift adaptation method. However, the retraining model takes time and is costly. Some companies with large models and massive datasets may be unable to afford it. Besides, the stationary model's performance may degrade over time. Thus, our model reuse preserves the model performance while keeping the maintenance time and cost low for companies with low budgets and close deadlines.

Researchers Preliminary study results show recurrent distribution patterns in time series datasets. Therefore, researchers can benefit from the approach and results of this study to find similar distribution patterns or to propose and develop tools that find similar data distributions in time series datasets and other types of datasets.

Furthermore, as we do not know the distribution of the actual upcoming data in the environment, it is critical to anticipate the distribution of the upcoming data in order to choose the right model to make predictions. Thus, researchers can benefit from our forecasting methods to find the distribution of their upcoming data. Moreover, they can propose new approaches to predict the distribution of the upcoming data. Finally, researchers and developers can provide visualization tools to monitor and compare the distributions of different time windows.

Besides, researchers can explore different forecasting and similarity measurement techniques to find which method works better and costs less for each data type.

Cloud Service Providers This paper proposes an improved version of the MLOps pipeline. To facilitate the use of this pipeline, cloud service providers (e.g. AWS, Azure) can implement APIs to automate the recurrent distribution and seasonality detection in the datasets. Secondly, they can provide our improved MLOps pipeline in their systems for their users so that developers can easily use it in their projects on the cloud.

Our approach mentioned the importance of storing the trained models and reusing them for similar data distributions in the future. Therefore, cloud service providers can benefit from this work's findings by facilitating model storage and reuse for their users.

MLOps Engineers We proposed an improved MLOps pipeline in this study. Therefore, MLOps engineers can benefit from our improved MLOps pipeline to modify their model maintenance approach. They can integrate our MLOp pipeline to reduce their maintenance costs and time.

8 Threats to Validity

This section explains the internal, external, construct, and conclusion validity of this study, along with the corresponding mitigation strategies.

8.1 Internal Validity Threats

One threat to the validity of this research is the effect of the methods used in this study on the results. We utilized two or more methods in each step to eliminate this threat. For example, to reduce the effect of the similarity measurement technique on the results, we used two different similarity metrics: Wasserstein distance and TVD. Another example is that we used two different learning algorithms to eliminate the effect of learning algorithms on our results. Therefore, the specific methods will not affect the generalizability of the results. Moreover, to find the proper segment length, we also used two methods to reduce each method's bias and effect. Finally, this study does not consider the storage cost because developers can save their models and data in their local storage.

Another threat to the validity of this research is finding the optimal segment length, which is key to the periodically updated model approach. Although we can test all the possible segment lengths for periodical retraining, it is costly and takes too much time. Therefore, it is not feasible to do that. To reduce the effort of finding the optimal segment length for 1) periodic training and 2) preliminary study and eliminate the risk of not considering all the possible values, we tested different values in the range of small segment lengths to large segment lengths.

8.2 External Validity Threats

The number of datasets used to evaluate the approach affects the generalizability of the results. To minimize this effect, we selected five widely used research datasets. We assume

that our findings from these four time series datasets can be generalized to other time series datasets. However, our tool should be tested on time series datasets in other domains.

Besides, SimReuse is designed to reuse previously trained models when their training data distribution closely resembles that of the forecast data. However, this assumption may not apply to some scenarios. For example, in a holiday context, indeed, SimReuse can still rely on existing models if the conditions during the holiday match the distribution represented in the training data. However, some holidays may impact the distribution of some time series datasets dramatically. For instance, Black Friday will impact the web traffic of the Amazon website. Thus, SimReuse may not be effective enough on holidays like Black Friday and Boxing Day to forecast the web traffic.

8.3 Construct Validity Threats

We use WD and TVD to quantify distribution similarity. These metrics may not fully capture complex characteristics of time-series distributions. To address this, we evaluate both metrics across five diverse datasets and report their individual effectiveness. This dual-metric evaluation ensures our findings are not dependent on a single similarity measurement technique.

9 Conclusion and Future Work

This study explores the seasonality and recurrent drift in the time series datasets and proposes an improved MLOps pipeline and a new ML model maintenance approach that achieves comparable ML model performance to the baselines, while keeping the maintenance time and cost low. We propose a model reuse approach to maintain ML models in a production environment.

After conducting a preliminary study, we realized that there are seasonal and recurrent distribution patterns in the studied time series datasets. We benefit from the recurrent distribution patterns to avoid unnecessary periodical model retraining. Instead, we utilize our previously trained models for inference on the future data segments with distributions similar to our existing models' training sets. Our evaluation results show that our model reuse approach performs comparably to the periodical and informed model retraining approaches, and it decreases the training time and cost by 87.5% and 11.5%, respectively. Therefore, companies, MLOps engineers and developers can benefit from our improved MLOps pipeline and our model reuse approach to save time, cost and resources while maintaining their ML models.

Future research directions include extending our study beyond time series datasets to explore whether the model reuse approach can also be applied effectively to other data types, such as streaming data.

Appendix

Tables 12, 13, 14, and 15 show the MSE and MAE performance results of the XGB and the GRU learning algorithms.

Table 12 Comparison of MSE of different model update approaches (learning algorithm: XGB)

Dataset	Similarity metric	Forecast approach	Stationary model MSE	Periodically updated model MSE	model reuse MSE	Informed retraining MSE	Random model MSE
NSW Electricity	WD	SA	16×10^{-4}	98×10^{-5}	85×10^{-5}	13×10^{-4}	34×10^{-3}
NSW Electricity	TVD	SA	16×10^{-4}	98×10^{-5}	49×10^{-5}	17×10^{-4}	34×10^{-3}
NSW Electricity	WD	ES	16×10^{-4}	98×10^{-5}	35×10^{-5}	94×10^{-5}	34×10^{-3}
NSW Electricity	TVD	ES	16×10^{-4}	98×10^{-5}	95×10^{-5}	82×10^{-5}	34×10^{-3}
Zurich Electricity	WD	SA	11×10^{-3}	67×10^{-4}	57×10^{-4}	99×10^{-4}	60×10^{-3}
Zurich Electricity	TVD	SA	11×10^{-3}	67×10^{-4}	76×10^{-4}	81×10^{-4}	60×10^{-3}
Zurich Electricity	WD	ES	11×10^{-3}	67×10^{-4}	78×10^{-4}	71×10^{-4}	60×10^{-3}
Zurich Electricity	TVD	ES	11×10^{-3}	67×10^{-4}	69×10^{-4}	63×10^{-4}	60×10^{-3}
Oil temperature	WD	SA	13×10^{-2}	59×10^{-4}	25×10^{-4}	83×10^{-4}	10×10^{-3}
Oil temperature	TVD	SA	13×10^{-2}	59×10^{-4}	35×10^{-3}	81×10^{-4}	10×10^{-3}
Oil temperature	WD	ES	13×10^{-2}	59×10^{-4}	24×10^{-4}	10×10^{-3}	10×10^{-3}
Oil temperature	TVD	ES	13×10^{-2}	59×10^{-4}	88×10^{-4}	10×10^{-3}	10×10^{-3}
Weather	WD	SA	13×10^{-3}	24×10^{-5}	13×10^{-5}	75×10^{-5}	37×10^{-3}
Weather	TVD	SA	13×10^{-3}	24×10^{-5}	11×10^{-4}	43×10^{-5}	37×10^{-3}
Weather	WD	ES	13×10^{-3}	24×10^{-5}	22×10^{-4}	51×10^{-5}	37×10^{-3}
Weather	TVD	ES	13×10^{-3}	24×10^{-5}	29×10^{-4}	35×10^{-5}	37×10^{-3}
NIFTY-50	WD	SA	31×10^{-4}	14×10^{-4}	20×10^{-4}	25×10^{-4}	30×10^{-3}
NIFTY-50	TVD	SA	31×10^{-4}	14×10^{-4}	32×10^{-4}	60×10^{-4}	30×10^{-3}
NIFTY-50	WD	ES	31×10^{-4}	14×10^{-4}	35×10^{-4}	64×10^{-4}	30×10^{-3}
NIFTY-50	TVD	ES	31×10^{-4}	14×10^{-4}	35×10^{-4}	43×10^{-4}	30×10^{-3}

Table 13 Comparison of MAE of different model update approaches (learning algorithm: XGB)

Dataset	Similarity metric	Forecast approach	Stationary model MAE	periodically updated model MAE	model reuse MAE	Informed retraining MAE	Random model MAE
NSW Electricity	WD	SA	25×10^{-3}	88×10^{-4}	94×10^{-4}	11×10^{-3}	10×10^{-2}
NSW Electricity	TVD	SA	25×10^{-3}	88×10^{-4}	89×10^{-4}	12×10^{-3}	10×10^{-2}
NSW Electricity	WD	ES	25×10^{-3}	88×10^{-4}	81×10^{-4}	93×10^{-4}	10×10^{-2}
NSW Electricity	TVD	ES	25×10^{-3}	88×10^{-4}	10×10^{-3}	88×10^{-4}	10×10^{-2}
Zurich Electricity	WD	SA	80×10^{-3}	59×10^{-3}	47×10^{-3}	73×10^{-3}	19×10^{-2}
Zurich Electricity	TVD	SA	80×10^{-3}	59×10^{-3}	57×10^{-3}	65×10^{-3}	19×10^{-2}
Zurich Electricity	WD	ES	80×10^{-3}	59×10^{-3}	59×10^{-3}	61×10^{-3}	19×10^{-2}
Zurich Electricity	TVD	ES	80×10^{-3}	59×10^{-3}	48×10^{-3}	57×10^{-3}	19×10^{-2}
Oil temperature	WD	SA	32×10^{-2}	59×10^{-3}	24×10^{-3}	71×10^{-3}	81×10^{-3}
Oil temperature	TVD	SA	32×10^{-2}	59×10^{-3}	85×10^{-3}	69×10^{-3}	81×10^{-3}
Oil temperature	WD	ES	32×10^{-2}	59×10^{-3}	21×10^{-3}	80×10^{-3}	81×10^{-3}
Oil temperature	TVD	ES	32×10^{-2}	59×10^{-3}	62×10^{-3}	80×10^{-3}	81×10^{-3}
Weather	WD	SA	65×10^{-3}	45×10^{-4}	28×10^{-4}	87×10^{-4}	15×10^{-2}
Weather	TVD	SA	65×10^{-3}	45×10^{-4}	93×10^{-4}	59×10^{-4}	15×10^{-2}
Weather	WD	ES	65×10^{-3}	45×10^{-4}	15×10^{-3}	66×10^{-4}	15×10^{-2}
Weather	TVD	ES	65×10^{-3}	45×10^{-4}	15×10^{-3}	56×10^{-4}	15×10^{-2}
NIFTY-50	WD	SA	37×10^{-3}	17×10^{-3}	23×10^{-3}	30×10^{-3}	10×10^{-2}
NIFTY-50	TVD	SA	37×10^{-3}	17×10^{-3}	36×10^{-3}	57×10^{-3}	10×10^{-2}
NIFTY-50	WD	ES	37×10^{-3}	17×10^{-3}	32×10^{-3}	68×10^{-3}	10×10^{-2}
NIFTY-50	TVD	ES	37×10^{-3}	17×10^{-3}	26×10^{-3}	50×10^{-3}	10×10^{-2}

Table 14 Comparison of MSE of different model update approaches (learning algorithm: GRU)

Dataset	Similarity metric	Forecast approach	Stationary model MSE	periodically updated model MSE	model reuse MSE	Informed retraining MSE	Random model MSE
NSW Electricity	WD	SA	24×10^{-4}	14×10^{-4}	13×10^{-4}	14×10^{-4}	34×10^{-3}
NSW Electricity	TVD	SA	24×10^{-4}	14×10^{-4}	11×10^{-4}	16×10^{-4}	34×10^{-3}
NSW Electricity	WD	ES	24×10^{-4}	13×10^{-4}	11×10^{-4}	16×10^{-4}	34×10^{-3}
NSW Electricity	TVD	ES	24×10^{-4}	14×10^{-4}	15×10^{-4}	13×10^{-4}	34×10^{-3}
Zurich Electricity	WD	SA	13×10^{-3}	11×10^{-3}	13×10^{-3}	13×10^{-3}	59×10^{-3}
Zurich Electricity	TVD	SA	13×10^{-3}	11×10^{-3}	21×10^{-3}	11×10^{-3}	59×10^{-3}
Zurich Electricity	WD	ES	13×10^{-3}	11×10^{-3}	13×10^{-3}	13×10^{-3}	59×10^{-3}
Zurich Electricity	TVD	ES	13×10^{-3}	11×10^{-3}	14×10^{-3}	10×10^{-3}	59×10^{-3}
Oil temperature	WD	SA	70×10^{-3}	60×10^{-4}	52×10^{-4}	10×10^{-3}	10×10^{-3}
Oil temperature	TVD	SA	70×10^{-3}	60×10^{-4}	28×10^{-3}	92×10^{-4}	10×10^{-3}
Oil temperature	WD	ES	70×10^{-3}	61×10^{-4}	52×10^{-4}	72×10^{-4}	10×10^{-3}
Oil temperature	TVD	ES	70×10^{-3}	61×10^{-4}	10×10^{-3}	78×10^{-4}	10×10^{-3}
Weather	WD	SA	10×10^{-4}	16×10^{-5}	17×10^{-5}	17×10^{-5}	37×10^{-3}
Weather	TVD	SA	10×10^{-4}	17×10^{-5}	24×10^{-5}	21×10^{-5}	37×10^{-3}
Weather	WD	ES	10×10^{-4}	15×10^{-5}	29×10^{-5}	14×10^{-5}	37×10^{-3}
Weather	TVD	ES	10×10^{-4}	13×10^{-5}	41×10^{-5}	13×10^{-5}	37×10^{-3}
NIFTY-50	WD	SA	26×10^{-4}	97×10^{-5}	12×10^{-4}	16×10^{-4}	30×10^{-3}
NIFTY-50	TVD	SA	26×10^{-4}	19×10^{-4}	31×10^{-4}	32×10^{-4}	30×10^{-3}
NIFTY-50	WD	ES	26×10^{-4}	12×10^{-4}	26×10^{-4}	27×10^{-4}	30×10^{-3}
NIFTY-50	TVD	ES	26×10^{-4}	94×10^{-5}	10×10^{-4}	15×10^{-4}	30×10^{-3}

Table 15 Comparison of MAE of different model update approaches (learning algorithm: GRU)

Dataset	Similarity metric	Forecast approach	Stationary model MAE	periodically updated model MAE	model reuse MAE	Informed retraining MAE	Random model MAE
NSW Electricity	WD	SA	34×10^{-3}	16×10^{-3}	16×10^{-3}	17×10^{-3}	10×10^{-2}
NSW Electricity	TVD	SA	34×10^{-3}	16×10^{-3}	17×10^{-3}	18×10^{-3}	10×10^{-2}
NSW Electricity	WD	ES	34×10^{-3}	16×10^{-3}	16×10^{-3}	17×10^{-3}	10×10^{-2}
NSW Electricity	TVD	ES	34×10^{-3}	17×10^{-3}	19×10^{-3}	17×10^{-3}	10×10^{-2}
Zurich Electricity	WD	SA	90×10^{-3}	83×10^{-3}	88×10^{-3}	90×10^{-3}	19×10^{-2}
Zurich Electricity	TVD	SA	90×10^{-3}	84×10^{-3}	11×10^{-2}	81×10^{-3}	19×10^{-2}
Zurich Electricity	WD	ES	90×10^{-3}	83×10^{-3}	88×10^{-3}	90×10^{-3}	19×10^{-2}
Zurich Electricity	TVD	ES	90×10^{-3}	84×10^{-3}	94×10^{-3}	81×10^{-3}	19×10^{-2}
Oil temperature	WD	SA	22×10^{-2}	60×10^{-3}	54×10^{-3}	78×10^{-3}	80×10^{-3}
Oil temperature	TVD	SA	22×10^{-2}	60×10^{-3}	10×10^{-2}	74×10^{-3}	80×10^{-3}
Oil temperature	WD	ES	22×10^{-2}	60×10^{-3}	54×10^{-3}	67×10^{-3}	80×10^{-3}
Oil temperature	TVD	ES	22×10^{-2}	60×10^{-3}	78×10^{-3}	69×10^{-3}	80×10^{-3}
Weather	WD	SA	21×10^{-3}	94×10^{-4}	92×10^{-4}	95×10^{-4}	15×10^{-2}
Weather	TVD	SA	21×10^{-3}	99×10^{-4}	10×10^{-3}	10×10^{-3}	15×10^{-2}
Weather	WD	ES	21×10^{-3}	91×10^{-4}	11×10^{-3}	89×10^{-4}	15×10^{-2}
Weather	TVD	ES	21×10^{-3}	88×10^{-4}	13×10^{-3}	86×10^{-4}	15×10^{-2}
NIFTY-50	WD	SA	46×10^{-3}	22×10^{-3}	22×10^{-3}	28×10^{-3}	10×10^{-2}
NIFTY-50	TVD	SA	46×10^{-3}	26×10^{-3}	45×10^{-3}	45×10^{-3}	10×10^{-2}
NIFTY-50	WD	ES	46×10^{-3}	23×10^{-3}	37×10^{-3}	43×10^{-3}	10×10^{-2}
NIFTY-50	TVD	ES	46×10^{-3}	22×10^{-3}	21×10^{-3}	30×10^{-3}	10×10^{-2}

Author Contributions Forough Majidi 1) proposed the idea and approach, and 2) contributed to writing the draft and resolving the writing issues. Foutse Khomh, Heng Li, and Amin Nikanjam contributed to 1) improving the idea and approach and 2) improving methods used in the study. They also contributed to improving the manuscript by changing the structure of the paper, detecting the writing issues, and proofreading it.

Funding This work is funded by the following organizations and companies: Fonds de Recherche du Quebec (FRQ), Natural Sciences and Engineering Research Council of Canada (NSERC), Mitacs, the Canadian Institute for Advanced Research (CIFAR), and the Canada Research Chairs Program. However, the findings and opinions expressed in this paper are those of the authors and do not necessarily represent or reflect those organizations.

Data Availability All data used for analysis, as well as the scripts and the analysis results, are provided in Majidi (2024).

Declarations

Ethical Approval Not applicable. This study does not involve any human participants or animals. Therefore, we do not require ethical approval.

Informed Consent Not applicable. This study does not involve a survey or human participants. Therefore, we do not require informed consent.

Conflicts of Interest The authors declared that they have no conflict of interest.

Clinical Trial Number Not applicable

References

Abbasi A, Javed AR, Chakraborty C, Nebhen J, Zehra W, Jalil Z (2021) Elstream: An ensemble learning approach for concept drift detection in dynamic social big data stream learning. IEEE Access 9:66408–66419

Amazon Web Services (2024a) Amazon sagemaker pricing. https://aws.amazon.com/sagemaker/pricing/, Accessed 2024-09-07

Amazon Web Services (2024b) Sagemaker clarify data bias metric: Total variation distance. https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-data-bias-metric-total-variation-distance.html, Accessed 2024-11-02

Amazon Web Services (2025) Amazon s3 pricing. https://aws.amazon.com/s3/pricing/. Accessed 17 Apr 2025

Amershi S, Begel A, Bird C, DeLine R, Gall H, Kamar E, Nagappan N, Nushi B, Zimmermann T (2019) Software engineering for machine learning: A case study. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp 291–300, https://doi.org/10.1109/ICSE-SEIP.2019.00042

Analytics Vidhya (2020) Sales forecasting — an lstm use case. https://medium.com/analytics-vidhya/sales-forecasting-an-lstm-use-case-2028c81d90eb. Accessed 24 July 2025

Aubin-Frankowski PC, Rudi A (2024) Approximation of optimization problems with constraints through kernel sum-of-squares. Optimization pp 1–26

Backblaze (2024) Hard drive test data. https://www.backblaze.com/cloud-storage/resources/hard-drive-test-data. Accessed 13 July 2024

Bayram F, Ahmed BS, Kassler A (2022) From concept drift to model degradation: An overview on performance-aware drift detectors. Knowl-Based Syst 245:108632

Box GE, Jenkins GM, Reinsel GC, Ljung GM (2015) Time series analysis: forecasting and control. Wiley

Brynjolfsson E, Mcafee A (2017) Artificial intelligence, for real. Harv Bus Rev 1:1–31

Brzezinski D, Stefanowski J (2013) Reacting to different types of concept drift: The accuracy updated ensemble algorithm. IEEE transactions on neural networks and learning systems 25(1):81–94

Cano A, Krawczyk B (2020) Kappa updated ensemble for drifting data stream mining. Mach Learn 109(1):175–218

Chung J, Gulcehre C, Cho K, Bengio Y (2014) Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv preprint arXiv:1412.3555
CircleCI (2024) Top 7 challenges of ml model development. https://circleci.com/blog/top-7-challenges-of-ml-model-development/challenge-7-continuous-training. Accessed 22 Nov 2024
Clarifyio Contributors (2024) Time series data. https://www.clarify.io/learn/time-series-data. Accessed 26 Nov 2024
Das A, Kong W, Sen R, Zhou Y (2023) A decoder-only foundation model for time-series forecasting. arXiv preprint arXiv:2310.10688
Dongre PB, Malik LG (2014) A review on real time data stream classification and adapting to various concept drift scenarios. In: 2014 IEEE International advance computing conference (IACC), IEEE, pp 533–537
Gama J, Medas P, Castillo G, Rodrigues P (2004) Learning with drift detection. In: Advances in Artificial Intelligence–SBIA 2004: 17th Brazilian Symposium on Artificial Intelligence, Sao Luis, Maranhao, Brazil, September 29-Ocotber 1, 2004. Proceedings 17, Springer, pp 286–295
Gama J, Zliobaite I, Bifet A, Pechenizkiy M, Bouchachia A (2014) A survey on concept drift adaptation. ACM Comput Surv (CSUR) 46(4):1–37
Gers FA, Schmidhuber J, Cummins F (2000) Learning to forget: Continual prediction with lstm. Neural Comput 12(10):2451–2471
Günay D (2021) Random forest. https://medium.com/@denizgunay/random-forest-af5bde5d7e1e. Accessed 04 July 2024
Goodfellow I, Bengio Y, Courville A, Bengio Y (2016) Deep learning, vol 1. MIT press Cambridge
Google Cloud (2024) Mlops: Continuous delivery and automation pipelines in machine learning. https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning. Accessed 07 Sept 2024
Harel M, Mannor S, El-Yaniv R, Crammer K (2014) Concept drift detection through resampling. In: International conference on machine learning, PMLR, pp 1009–1017
Hepworth S (2022) Mlops: Productionising ml models at google scale. https://medium.com/google-cloud/mlops-productionising-ml-models-at-google-scale-e78949c20e0a. Accessed 26 Aug 2024
Herzen J, Lässig F, Piazzetta SG, Neuer T, Tafti L, Raille G, Pottelbergh TV, Pasieka M, Skrodzki A, Huguenin N, Dumonal M, KoÅcisz J, Bader D, Gusset F, Benheddi M, Williamson C, Kosinski M, Petrik M, Grosch G (2022) Darts: User-friendly modern machine learning for time series. J Mach Learn Res 23(124):1–6. http://jmlr.org/papers/v23/21-1177.html
Hochreiter S, Schmidhuber J (1997) Long short-term memory. Neural Comput 9(8):1735–1780
Hoens TR, Polikar R, Chawla NV (2012) Learning from streaming data with concept drift and imbalance: an overview. Prog in Artif Intell 1:89–101
Hopsworks (2023) Mlops to ml systems with fti pipelines. https://www.hopsworks.ai/post/mlops-to-ml-systems-with-fti-pipelines. Accessed 24 Oct 2024
Hyndman RJ, Athanasopoulos G (2018) Forecasting: principles and practice. OTexts
IBM (2024) Random forest. https://www.ibm.com/topics/random-forest. Accessed 04 July 2024
Jafari M, Majidi F, Heydarnoori A (2025) Prioritizing app reviews for developer responses on google play. arXiv preprint arXiv:2502.01520
Jain VR, Gupta M, Singh RM (2018) Analysis and prediction of individual stock prices of financial sector companies in nifty50. Int J Inform Eng Electron Bus 13(2):33
Jin M, Wang S, Ma L, Chu Z, Zhang JY, Shi X, Chen PY, Liang Y, Li YF, Pan S, et al (2023) Time-llm: Time series forecasting by reprogramming large language models. arXiv preprint arXiv:2310.01728
Levin DA, Peres Y (2017) Markov chains and mixing times, vol 107. Am Math Soc
Lyu Y, Li H, Sayagh M, Jiang ZM, Hassan AE (2021) An empirical study of the impact of data splitting decisions on the performance of aiops solutions. ACM Trans Softw Eng Methodol (TOSEM) 30(4):1–38
Lyu Y, Rajbahadur GK, Lin D, Chen B, Jiang ZM (2021) Towards a consistent interpretation of aiops models. ACM Trans Softw Eng Methodol (TOSEM) 31(1):1–38
Lyu Y, Li H, Ming Z, Hassan AE et al (2023) Assessing the maturity of model maintenance techniques for aiops solutions. arXiv preprint arXiv:2311.03213
Lyu Y, Li H, Jiang ZM, Hassan AE (2024) On the model update strategies for supervised learning in aiops solutions. ACM Trans Softw Eng Methodol 33(7):1–38
Majidi F (2024) Ml-maintenance replication package. https://github.com/ForoughMajidi1995/Maintenance
Majidi F, Openja M, Khomh F, Li H (2022) An empirical study on the usage of automated machine learning tools. In: 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, pp 59–70
Mallick A, Hsieh K, Arzani B, Joshi G (2022) Matchmaker: Data drift mitigation in machine learning for large-scale systems. Proceedings of machine learning and systems 4:77–94
Max Planck Institute for Biogeochemistry (2024) Weather data. https://www.bgc-jena.mpg.de/wetter/. Accessed 02 July 2024

Minku LL, Yao X (2011) Ddd: A new ensemble approach for dealing with concept drift. IEEE Trans Knowl and Data Eng 24(4):619–633
Minku LL, White AP, Yao X (2009) The impact of diversity on online ensemble learning in the presence of concept drift. IEEE Trans Knowl Data Eng 22(5):730–742
Neto AF, Canuto AM (2021) Eocd: An ensemble optimization approach for concept drift applications. Inform Sci 561:81–100
Nishida K, Yamauchi K (2007) Detecting concept drift using statistical testing. In: International conference on discovery science, Springer, pp 264–269
NVIDIA (2024) Xgboost. https://www.nvidia.com/en-us/glossary/xgboost/. Accessed 04 July 2024
Openja M, Majidi F, Khomh F, Chembakottu B, Li H (2022) Studying the practices of deploying machine learning projects on docker. In: Proceedings of the 26th international conference on evaluation and assessment in software engineering, pp 190–200
Patil PS, Kukreja V, Upadhyay D, Manwal M, Sharma R (2024) A long short-term memory technique for nifty50 stocks price prediction. In: 2024 Asia pacific conference on innovation in technology (APCIT), IEEE, pp 1–5
Poenaru-Olaru L, Cruz L, van Deursen A, Rellermeyer JS (2022) Are concept drift detectors reliable alarming systems?-a comparative study. In: 2022 IEEE International conference on big data (Big Data), IEEE, pp 3364–3373
Poenaru-Olaru L, Cruz L, Rellermeyer JS, Van Deursen A (2023) Maintaining and monitoring aiops models against concept drift. In: 2023 IEEE/ACM 2nd international conference on AI engineering–software engineering for AI (CAIN), IEEE, pp 98–99
Poenaru-Olaru L, Karpova N, Cruz L, Rellermeyer JS, van Deursen A (2024) Is your anomaly detector ready for change? adapting aiops solutions to the real world. In: Proceedings of the IEEE/ACM 3rd international conference on AI engineering-software engineering for AI, pp 222–233
Reddit Contributors (2024) Eli5: What does p(x), p(x|y), and p(x,y) mean? https://www.reddit.com/r/explainlikeimfive/comments/u3lnut/eli5what_does_px_px_y_and_px_y_mean/?rdt=46557. Accessed 26 Nov 2024
RunAI Contributors (2024) Machine learning operations guide. https://www.run.ai/guides/machine-learning-operations. Accessed 26 Nov 2024
Scikit-learn Developers (2024) Minmaxscaler. https://scikit-learn.org/0.15/modules/generated/sklearn.preprocessing.MinMaxScaler.html. Accessed 03 July 2024
Sculley D, Holt G, Golovin D, Davydov E, Phillips T, Ebner D, Chaudhary V, Young M, Crespo JF, Dennison D (2015) Hidden technical debt in machine learning systems. Adv Neural Inform Process Syst 28
Shumway RH, Stoffer DS (2017) Time Series Analysis and Its Applications: With R Examples, 4th edn. Springer Texts in Statistics, Springer, Cham. https://doi.org/10.1007/978-3-319-52452-8
Simplilearn (2023) Exponential smoothing for time series forecasting in python. https://www.simplilearn.com/exponential-smoothing-for-time-series-forecasting-in-python-article. Accessed 02 Nov 2024
Spotify Engineering (2023) Large-scale generation of ml podcast previews at spotify with google dataflow. https://engineering.atspotify.com/2023/04/large-scale-generation-of-ml-podcast-previews-at-spotify-with-google-dataflow/. Accessed 24 Oct 2024
Street WN, Kim Y (2001) A streaming ensemble algorithm (sea) for large-scale classification. In: Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining, pp 377–382
Tahmasbi A, Jothimurugesan E, Tirthapura S, Gibbons PB (2020) Driftsurf: A risk-competitive learning algorithm under concept drift. arXiv preprint arXiv:2003.06508
The Pandas Development Team (2024) Pandas to datetime documentation. https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html. Accessed 04 July 2024
Timescale Contributors (2024) Time series analysis and forecasting with python. https://www.timescale.com/learn/time-series-analysis-and-forecasting-with-python. Accessed 26 Nov 2024
UbiOps Contributors (2024) An introduction to model drift in machine learning. https://ubiops.com/an-introduction-to-model-drift-in-machine-learning/. Accessed 26 Nov 2024
Undy B (2024) The art and science of algorithm selection in machine learning. https://www.linkedin.com/pulse/art-science-algorithm-selection-machine-learning-bryce-undy-3uryc/, linkedIn Pulse article
Unit8co (2024) Darts datasets api documentation. https://unit8co.github.io/darts/generated_api/darts.datasets.html. Accessed 02 July 2024
Vidhya A (2018) An end-to-end guide to understand the math behind xgboost. https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-xgboost/. Accessed 04 July 2024
Wang H, Fan W, Yu PS, Han J (2003) Mining concept-drifting data streams using ensemble classifiers. In: Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining, pp 226–235

Wang S, Minku LL, Ghezzi D, Caltabiano D, Tino P, Yao X (2013) Concept drift detection for online class imbalance learning. In: The 2013 International Joint Conference on Neural Networks (IJCNN), IEEE, pp 1–10
Widmer G, Kubat M (1996) Learning in the presence of concept drift and hidden contexts. Mach Learn 23:69–101
Wikipedia Contributors (2024a) Mean squared error. https://en.wikipedia.org/wiki/Mean_squared_error. Accessed 26 Nov 2024
Wikipedia Contributors (2024b) Random forest. https://en.wikipedia.org/wiki/Random_forest. Accessed 04 July 2024
Wikipedia Contributors (2024c) Total variation distance of probability measures. https://en.wikipedia.org/wiki/Total_variation_distance_of_probability_measures. Accessed 26 Nov 2024
Wikipedia Contributors (2024d) Wasserstein metric. https://en.wikipedia.org/wiki/Wasserstein_metric. Accessed 09 July 2024
Wilkes J (2020) Google cluster-usage traces v3. technical report. Google Inc, Mountain View, CA, USA
Willmott CJ, Matsuura K (2005) Advantages of the mean absolute error (mae) over the root mean square error (rmse) in assessing average model performance. Climate Res 30(1):79–82
Zeng A, Chen M, Zhang L, Xu Q (2023) Are transformers effective for time series forecasting? Proceedings of the AAAI conference on artificial intelligence 37:11121–11128
Zhang GP, Qi M (2005) Neural network forecasting for seasonal and trend time series. Eur J Oper Res 160(2):501–514
Zhou H (2024) Etdataset. https://github.com/zhouhaoyi/ETDataset. Accessed 02 July 2024
Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, Zhang W (2021) Informer: Beyond efficient transformer for long sequence time-series forecasting. Proceedings of the AAAI conference on artificial intelligence 35:11106–11115
Zliobaite I (2010) Learning under concept drift: an overview. arXiv preprint arXiv:1010.4784 41
Zliobaite I (2013) How good is the electricity benchmark for evaluating concept drift adaptation. arXiv preprint arXiv:1301.3524
Zürich S (2022) Stromabgabe nach netzebenen (ab 2012). https://data.stadt-zuerich.ch/dataset/ewz_stromabgabe_netzebenen_stadt_zuerich. Accessed 02 July 2024
Zürich S (2025) Meteodaten stundenmittelwerte. https://data.stadt-zuerich.ch/dataset/ugz_meteodaten_stundenmittelwerte. Accessed 02 July 2024

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Forough Majidi[1] · Foutse Khomh[1] · Heng Li[1] · Amin Nikanjam[1]

✉ Forough Majidi
forough.majidi@polymtl.ca

Foutse Khomh
foutse.khomh@polymtl.ca

Heng Li
heng.li@polymtl.ca

Amin Nikanjam
amin.nikanjam@polymtl.ca

[1] Polytechnique Montréal, Montréal, Québec, Canada